



Introducción a Redes Neuronales y DeepLearning



Dr. Nicolás Gálvez Ramírez
Dr. Patricio Olivares Roncagliolo

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

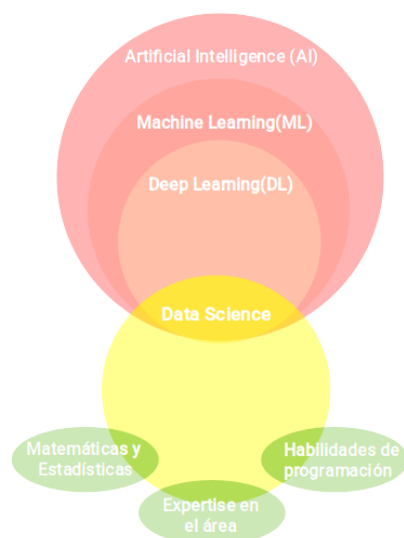
Fuentes:

"Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow"

"Stanford - CS229 Notes"

¿Qué es Machine Learning?

- Machine Learning (de ahora en adelante, **ML**), se define como el conjunto de técnicas a través de los cuales un computador puede aprender de los datos.



¿Por qué utilizar Machine Learning?

Ej. Creación de un filtro de spam para correos electrónicos

- Aproximación de ML

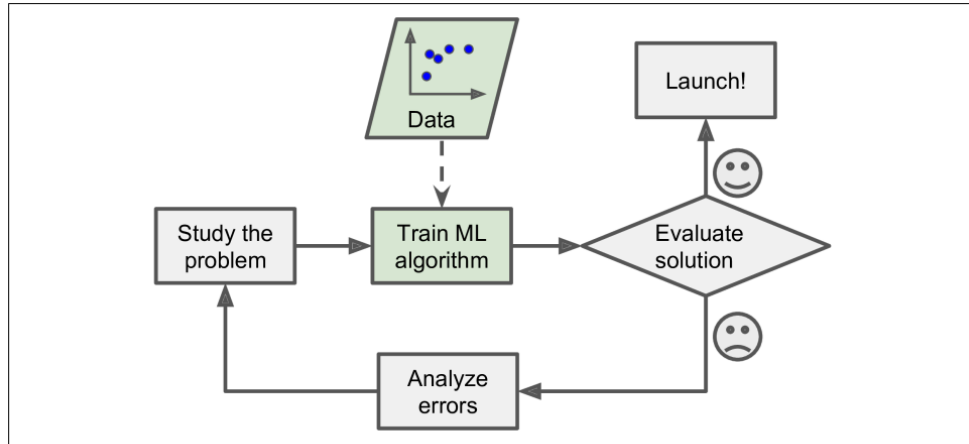


Figure 1-2. Machine Learning approach

Ej. Creación de un filtro de spam para correos electrónicos

- Aproximación tradicional

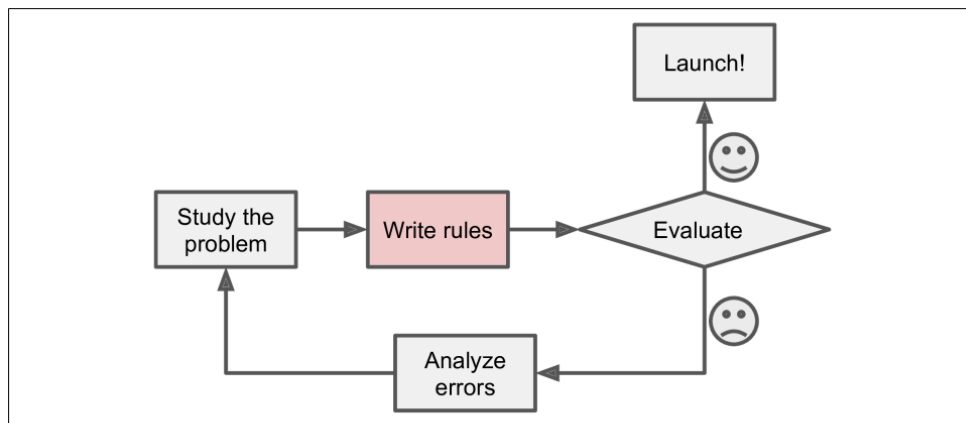


Figure 1-1. The traditional approach

Ej. Creación de un filtro de spam para correos electrónicos

- Aproximación de ML

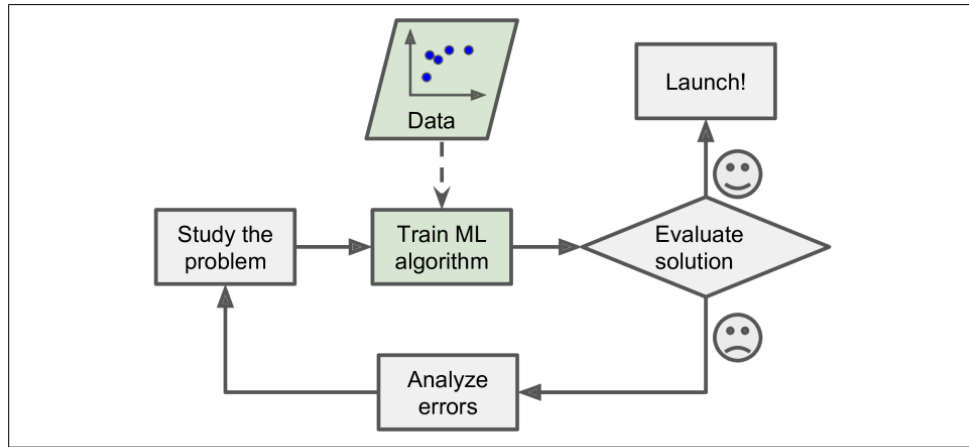


Figure 1-2. Machine Learning approach

Bibliotecas para ML python



Scikit-Learn



- Biblioteca para ML de código libre

- Biblioteca para Python
- Soporta múltiples algoritmos tanto supervisados como no supervisados
- Si está trabajando en su entorno local, previo a su importación debe ser **instalada**
 - `pip install scikit-learn` si usa `pip`
 - `conda install scikit-learn` si usa `conda`

Scikit-Learn: Utilización

Todo los objetos Scikit-Learn son **consistentes** entre sí

- **Estimadores:**
 - Método encargado de la **estimación** de los parámetros asociados al algoritmo.
 - Objetos Scikit-Learn incluyen el método *fit()*, cuya entrada es el set de datos utilizados para el cálculo de los parámetros.
- **Transformadores:**
 - Método encargado de la **transformación** de datos en base a los parámetros calculados por método *fit()*.
 - Objetos Scikit-Learn incluyen el método *transform()*, que recibe con los datos a transformar.
 - Método *fit_transform()* permite realizar ambos pasos con un solo método.
- **Predictores:**
 - Método encargado de hacer **predicciones** en base a datos de entrada.
 - Objetos Scikit-Learn incluyen el método *predict()*, para realizar predicciones en base a parámetros de entrada.

Cuáles de estos métodos están presentes, dependerá del tipo algoritmo a utilizar.

División en set de training y test

- Los modelos de Machine Learning intentan aprender patrones en los datos que le permitan hacer predicciones.
- Al momento de evaluar un modelo, se desea estimar qué tan bien es capaz de **generalizar** a partir de los datos usados en el proceso de aprendizaje. (¿Puedo hacer predicciones sobre nuevos datos a partir de lo ya aprendido?)
- Para ello, siempre es necesario dividir nuestro data set en dos grupos:
 - **Training set:** Es el subconjunto de datos que será utilizado para entrenar nuestro modelo.
 - **Test set:** Es el subconjunto de datos que será utilizado para evaluar la generalización de nuestro modelo.

```
In [1]: import os
os.environ['CUDA_VISIBLE_DEVICES'] = '-1'
```

```
In [2]: # Ejemplo de división training/test con scikit learn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Carga de dataset dentro de scikit learn
# Dataset de cáncer de mamas
cancer_data = load_breast_cancer()
X = cancer_data.data
y = cancer_data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print(X_train, y_train)
```

```
[[1.162e+01 1.818e+01 7.638e+01 ... 1.416e-01 2.660e-01 9.270e-02]
 [1.120e+01 2.937e+01 7.067e+01 ... 0.000e+00 1.566e-01 5.905e-02]
 [1.057e+01 1.832e+01 6.682e+01 ... 2.222e-02 2.699e-01 6.736e-02]
 ...
 [1.365e+01 1.316e+01 8.788e+01 ... 8.056e-02 2.380e-01 8.718e-02]
 [1.705e+01 1.908e+01 1.134e+02 ... 2.543e-01 3.109e-01 9.061e-02]
 [9.904e+00 1.806e+01 6.460e+01 ... 9.910e-02 2.614e-01 1.162e-01]] [1 1 1 1
1 1 1 0 1 1 0 0 0 1 1 0 0 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 0 0 1 1 1
1 1 1 1 1 1 0 1 1 1 0 0 0 1 1 1 0 0 1 0 1 1 0 0 0 1 1 1 1 0 1 1 1 0 0 0 1
1 1 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 1 0 0 0 0 1 1 0 1 1 1 1 0 0 0 1 0 0
0 0 1 1 1 0 1 1 1 1 1 1 1 1 0 0 1 1 0 1 0 0 0 0 1 0 0 0 1 1 1 1 1 1 1 0
1 1 1 0 1 1 1 0 0 1 0 0 1 0 1 1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 0 1 1 1
0 1 0 1 1 1 0 1 1 1 0 0 1 1 1 0 1 1 0 1 0 0 0 1 1 1 1 0 1 1 1 1 1 0 0 1
0 1 1 1 1 0 0 0 0 0 1 0 1 1 0 1 1 0 1 0 1 0 0 1 0 1 1 1 1 0 1 0 0 1 1 0
0 0 0 1 1 0 1 1 1 1 0 0 1 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 0 1 0 1 1 1 0 1
1 0 1 1 1 0 1 1 0 1 0 0 1 1 0 0 0 1 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 1 1
0 1 0 0 1 1 1 1 1 0 1 0 1 0 1 1 1 1 0 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1
0 1 0 1 1 0 0 0 1 0 1 1 1 1 1 0 0 1 1 0 1 0 0 0 0 1 0 1]
```

Tipos de errores

Para determinar el correcto funcionamiento de un algoritmo de clasificación, podemos preguntarnos

- ¿Cuántas veces acertamos la clasificación de una clase y cuántas veces no?

Los errores que puede cometer nuestro clasificador son de dos tipos:

Errores de Tipo 1: Falso Positivo

Clasificamos un elemento como *perteneciente* (positivo) a la clase **incorrectamente** (falso)



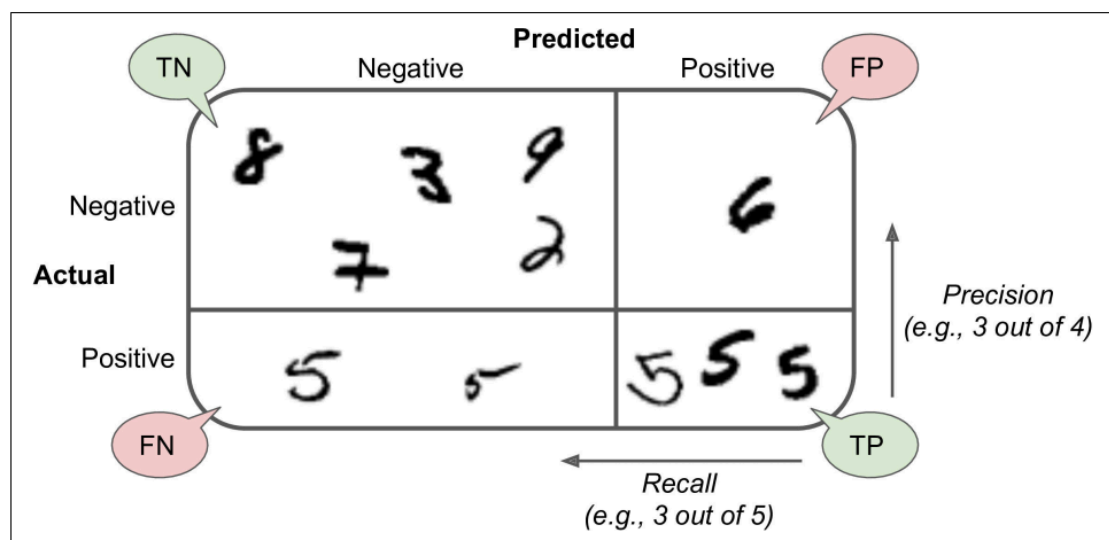
Errores de Tipo 2: Falso Negativo

Clasificamos un elemento como *no perteneciente* (negativo) a la clase **incorrectamente** (falso)



Matriz de confusión

Permite contar la cantidad de veces que instancias de la clase A son clasificadas como clase B



Fuente: "Hands-on Machine Learning with Scikit-Learn, Keras & Tensorflow", O'Reilly

Ejemplo

```
In [3]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
# Primera vez que se importa la biblioteca de aprendizaje automático scikit-
# No te preocupes si esto parece mucho código
from sklearn import metrics
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Ten en cuenta que es probable que el nombre del archivo sea diferente en t
filename = 'data/titanic.csv'
df = pd.read_csv(filename)
df.head()
```

```
Out[3]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8

```
In [4]: # Transforma la columna Sexo en una forma que no sea de texto
# Elijo cuatro funciones, podrías haber elegido otras
feature_cols = ['Pclass', 'Parch', 'Age', 'Sex']
# Convierta la columna Sexo en algo que pueda alimentar a un modelo
# Tiene
df['Sex'] = df.Sex.map({'male': 0,
                        'female': 1})
# Eliminar filas donde la edad es nan del conjunto de datos
df = df.loc[~df['Age'].isnull(), :]
# Crear matriz de características y vector objetivo
```

```

X = df.loc[:, feature_cols]
y = df['Survived']
# Divida los datos en conjuntos de prueba y entrenamiento
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    random_state = 0)

# Estandarizar datos
scaler = StandardScaler()
# Ajusta solo en el conjunto de entrenamiento
scaler.fit(X_train)
# Aplicar transformación tanto al conjunto de entrenamiento como al conjunto de prueba
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
# Crea una instancia de un modelo
logreg = LogisticRegression()
# Entrena el modelo con los datos
logreg.fit(X_train, y_train);
# predicciones de clase (no probabilidades predichas)
predictions = logreg.predict(X_test)
print(predictions)
print(y_test.tolist())

```

```

[1 1 1 0 1 0 0 0 0 0 0 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 0 1 0 0 0 0 1 0 0 1
 0 0 1 0 0 0 1 0 1 0 0 0 0 0 1 0 1 1 1 1 0 0 0 0 0 1 0 1 1 1 0 1 1 0 0 1 1
 1 1 1 0 0 0 0 0 0 1 1 0 0 0 1 0 1 0 1 0 1 0 1 1 0 0 1 1 1 0 1 1 1 0 1 0 0
 0 0 0 0 0 0 0 1 1 0 1 0 0 0 1 1 0 0 1 1 0 1 0 1 0 1 1 0 0 1 0 1 1 0 0 0 1
 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 0 1 1 0]
[0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0,
 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0,
 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0,
 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0,
 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0,
 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0,
 0, 1, 1, 0]

```

```

In [5]: # valores reales: y_test.values
# predicciones: predicciones del modelo
print("Total de elementos:", len(y_test.values))
cm = metrics.confusion_matrix(y_test.values, predictions)
print(cm)
print(cm.ravel())
tn, fp, fn, tp = cm.ravel()
print(tn, fp, fn, tp)

```

```

Total de elementos: 179
[[87 16]
 [17 59]]
[87 16 17 59]
87 16 17 59

```

Medidas para clasificación

Accuracy (exactitud)

- Fracción de casos correctamente clasificados sobre casos totales (¿Cuántas veces acerté?)

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- En nuestra matriz de confusión anterior, sería

$$\text{accuracy} = \frac{87 + 59}{87 + 16 + 17 + 59} = 0.8156 = 81.56\%$$

```
In [6]: score = logreg.score(X_test, y_test) # Esta línea, para este modelo, calcula
print("Score obtenido por Regresor", score)
manual_score = (tp + tn)/(tp + tn + fp + fn)
print("Score obtenido manualmente", manual_score)
```

Score obtenido por Regresor 0.8156424581005587

Score obtenido manualmente 0.8156424581005587

El accuracy es una excelente medida, pero presenta algunos problemas. Veamos un ejemplo para un "clasificador de 5s"

```
In [7]: # Ej. Modelo de clasificación de 5s
import numpy as np
# Datos de entrenamiento
datos = [1,2,1,3,4,2,5,3,4,6,5,4,2,7,9,8,9,6,4,5]
# Valor a predecir. 1 representa la existencia de un 5 y 0 la ausencia del mismo
valores_reales = [0,0,0,0,0,0,1,0,0,0,1,0,0,0,0,0,0,0,0,1]

# Crearemos el modelo "Nada es 5". Este modelo decide que ningún número es 5
# Emularemos las predicciones de este modelo que clasifica todo como
# "No 5" en la siguiente variable
predicciones = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0] # Crea una lista de 20 ceros
print(predicciones)
print(valores_reales)
# Nuestro modelo clasifica todo como NO 5
tn=17 # 17 son los números que no eran 5, por lo tanto nuestro modelo los clasifica correctamente
tp=0 # Como no clasificamos ningún 5, los TP son 0
fn=3 # Los 5s que clasificamos incorrectamente, están acá
fp=0 # No hicimos ninguna clasificación FP.

# Ejecuta esta celda y ve qué accuracy obtiene este modelo!
accuracy = (tn+tp)/(tn+tp+fn+fp)
print(accuracy*100)
```

[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

[0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1]

85.0

A pesar de haber creado un mal modelo, obtuvimos un alto accuracy. Esto se debe a que el accuracy hace un importante supuesto: Los datos de las clases positiva y negativa deben ser **COMPARABLES** en cantidad, lo cual no siempre ocurre.

Sensitivity/Recall/True Positive Rate (sensibilidad)

- Fracción de casos **pertenecientes** a una clase correctamente clasificados (¿Cuántas veces acerté al clasificar la clase X?)

$$\text{sensitivity} = \frac{TP}{TP + FN} = \frac{\text{Positivos correctamente clasificados}}{\text{Positivos reales totales}}$$

```
In [8]: # Ahora, calculemos la sensibilidad de nuestro modelo de  
# No 5s  
sensitivity = tp/(tp+fn)  
print(sensitivity*100)
```

0.0

Specificity/Selectivity/True Negative Rate (especificidad)

- Fracción de casos **no pertenecientes** a una clase correctamente clasificados (¿Cuántas veces acerté al clasificar casos fuera de la clase X?)

$$\text{specificity} = \frac{TN}{TN + FP}$$

```
In [9]: # Calculemos ahora la especificidad de nuestro modelo  
# No 5s  
specificity = (tn)/(tn + fp)  
print(specificity*100)
```

100.0

Precision/Positive predictions (precisión)

- Fracción de casos clasificados como **positivos** que fueron correctamente clasificados (¿Cuántos casos clasificados como X fueron acertados?)

$$\text{precision} = \frac{TP}{TP + FP} = \frac{\text{Casos correctamente clasificados}}{\text{Casos totales clasificados como Positivos}}$$

```
In [10]: # Calculemos la precision de nuestro modelo  
# No 5s  
#precision = tp/(tp+fp) # Como no hay ningún caso clasificado como positivo,  
#print(precision*100)
```

Redes neuronales (Neural Networks - NN)

- Las Redes Neuronales son un conjunto de modelos de Machine Learning.
- Estas se inspiran en el funcionamiento neurológico del cerebro humano.
- Las redes neuronales han sido utilizadas en múltiples y diferentes áreas de aplicación

Ejemplos de aplicaciones Deep Learning

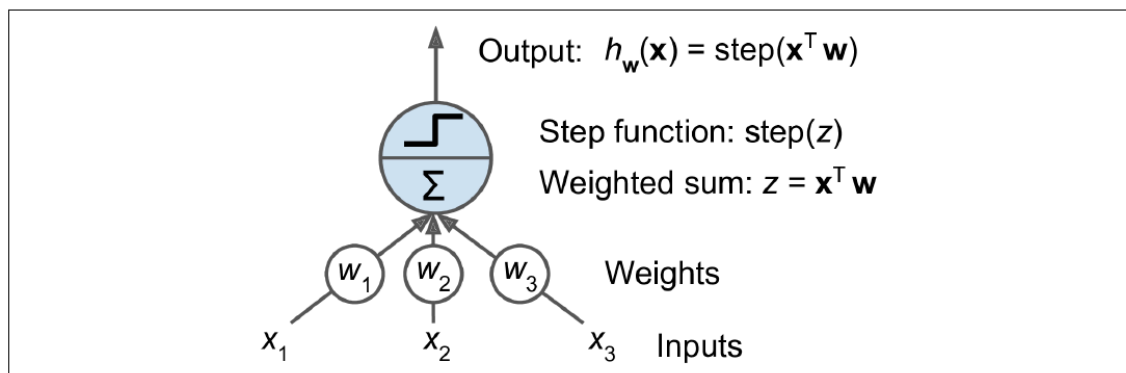
- [DeepMind](#)
- [Gaugan2](#)
- [YoloV3](#)
- [QuickDraw](#)
- [Rock-Paper-Scissors](#)
- [ChatGPT](#)
- Y una laaaarga [lista](#)

Perceptrón

- Es la red neuronal más sencilla cuya base es la TLU (Threshold Logic Unit)
- TLU es una neurona artificial que opera de la siguiente manera
 - Recibe una o varias entradas numéricas (datos de entrenamiento)
 $\mathbf{x} = (x_1, x_2, \dots, x_n)$
 - Cada entrada es ponderada por un peso $\mathbf{w} = (w_1x_1, w_2x_2, \dots, w_nx_n)$
 - Se calcula la suma ponderada de las entradas
 $z = w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{x}^T\mathbf{w}$
 - Al resultado de la suma ponderada se le aplica una función llamada función de activación $h_{\mathbf{w}}(\mathbf{x})$. El resultado corresponde a la predicción $\hat{y}$.

$$z = x_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{x}^T\mathbf{w}$$

$$\hat{y} = h_{\mathbf{w}}(\mathbf{x}) = h(z) = h(\mathbf{x}^T\mathbf{w})$$



```
In [11]: # Ejemplo de un perceptrón utilizando la biblioteca Scikit-Learn
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split

cancer_data = load_breast_cancer()
X = cancer_data.data
y = cancer_data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

per = Perceptron()
per.fit(X_train, y_train)
print("Training score:", per.score(X_train, y_train)) # Accuracy de entrenamiento
print("Test score:", per.score(X_test, y_test)) # Accuracy de testing
```

Training score: 0.9271356783919598
Test score: 0.8830409356725146

```
In [12]: print("Los valores reales:", y_test)
print("Predicciones:", per.predict(X_test))
```

Los valores reales: [0 1 1 0 0 0 1 0 1 0 1 1 0 0 1 0 0 1 1 1 1 1 0 0 1 1 1
0 0 1 0 1 1 0 1 1
1 1 1 1 0 1 1 1 1 1 1 1 0 1 1 0 0 1 1 0 1 1 1 0 0 0 1 1 1 1 0 1 1 0 0 0 0
1 1 1 1 0 1 1 1 0 1 0 1 1 0 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 1 1 1 0 1 0
1 0 1 1 0 0 1 1 0 1 1 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 1 1 1 0 0 0 0 0 1
1 1 1 1 0 1 0 1 1 0 1 1 0 0 1 1 0 1 1 1 1 0 1]
Predicciones: [1 1 1 1 1 1 1 0 1 0 1 1 0 0 1 0 0 1 1 1 1 1 0 0 0 1 0 0 0 1
0 1 1 0 1 1
1 1 0 0 0 1 1 1 1 1 1 1 1 1 0 0 1 1 0 1 1 0 0 0 0 1 1 1 1 0 1 1 0 0 0 0
1 1 1 1 0 1 1 1 0 1 0 1 1 0 0 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1
1 0 1 1 1 1 1 1 0 1 1 0 1 1 1 1 0 1 1 1 1 0 0 0 1 0 1 1 0 1 1 0 0 0 0 1
1 1 1 1 0 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 0 0 1]

Gradient descent

- Al entrenar una neurona, lo que se intenta determinar son los **pesos** $\mathbf{w}$ tales que la predicción $h_{\mathbf{w}}(\mathbf{x})$ sea lo más cercana al valor esperado.
- Considere el Suma de Errores Cuadrático (SSE) entre la salida de la neurona ($\hat{y}$) y el valor real esperado (y).

$$J(\mathbf{w}) = \text{SSE} = \frac{1}{2} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2 = \frac{1}{2} \sum_{i=1}^m \left(h(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

donde $\mathbf{X}$ es la matriz de datos de entrada (datos de entrenamiento), $\mathbf{w}$ es el vector de pesos, $\mathbf{y}$ es el vector de salidas esperadas, m es el total de muestras (datos de entrenamiento).

Al necesitarse que los valores de $\hat{y}$ e y sean los más cercanos posibles, el SSE puede ser utilizado como medida. Por lo tanto, **debemos ajustar los valores de w de tal manera que el SSE sea el menor posible.**

$$\min_{\mathbf{w}} J(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^m \left(h(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

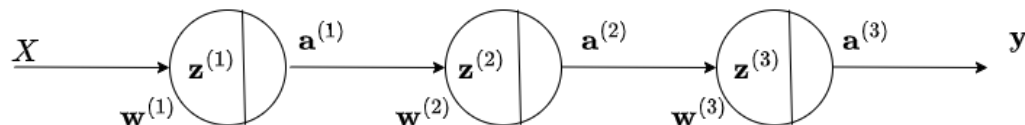
- El algoritmo de descenso del gradiente o Gradient Descent, permite calcular el mínimo local de una función
- Se define a través de la siguiente regla de actualización para cada elemento w_j perteneciente al vector de pesos $\mathbf{w}$.

$$w_j := w_j - \alpha \nabla_{w_j} J(\mathbf{w})$$

- Gradient Descent se utiliza para actualizar los pesos de la neurona al momento de entrenar.
- Actualiza el valor de cada peso w_j utilizando el gradiente de la función de costo definida anteriormente como base para la actualización (avanzo por el lugar que minimice el error de $J(\mathbf{w})$)

Back Propagation

- Back Propagation es un algoritmo que permite el cálculo de los gradientes requeridas por Gradient Descent utilizando la regla de la cadena.
- Estas derivadas se "propagan hacia atrás" de manera que el cálculo de una derivada se puede reutilizar para múltiples derivadas que dependan de ella por la regla de la cadena.
- Considere la siguiente red neuronal simple:



- Calculando la función de costo para una instancia, se obtiene lo siguiente:

$$J = (\mathbf{a}^{(3)} - y)$$

Donde $\mathbf{a}^{(3)}$ se puede definir de la siguiente manera:

$$\mathbf{a}^{(3)} = \mathbf{h}^{(3)}(\mathbf{z}^{(3)}) = \mathbf{h}^{(3)}(\mathbf{a}^{(2)} \cdot \mathbf{w}^{(3)})$$

donde $\mathbf{h}^{(i)}$ representa la función de activación de la capa i

- A continuación calcularemos cómo varía la función de costo (error) cuando modifico $\mathbf{w}^{(3)}$. Por regla de la cadena, se tiene

$$\frac{\partial J}{\partial \mathbf{w}^{(3)}} = \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{w}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial J}{\partial \mathbf{a}^{(3)}}$$

- Calculando cada derivada

$$\frac{\partial J}{\partial \mathbf{a}^{(3)}} = \frac{\partial}{\partial \mathbf{a}^{(3)}} (\mathbf{a}^{(3)} - \mathbf{y})^2 = 2(\mathbf{a}^{(3)} - \mathbf{y})$$

$$\frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} = \frac{\partial}{\partial \mathbf{z}^{(3)}} \mathbf{h}^{(3)}(\mathbf{z}^{(3)}) = \mathbf{h}'^{(3)}(\mathbf{z}^{(3)})$$

$$\frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{w}^{(3)}} = \frac{\partial}{\partial \mathbf{w}^{(3)}} (\mathbf{a}^{(2)} \mathbf{w}^{(3)}) = \mathbf{a}^{(2)}$$

- Todas las derivadas anteriores son fácilmente calculables, excepto $\mathbf{h}'^{(3)}$, pues depende de la función de activación escogida.
- Veamos qué ocurre cuando calculamos cómo varía la función de costo cuando modificamos $\mathbf{w}^{(2)}$

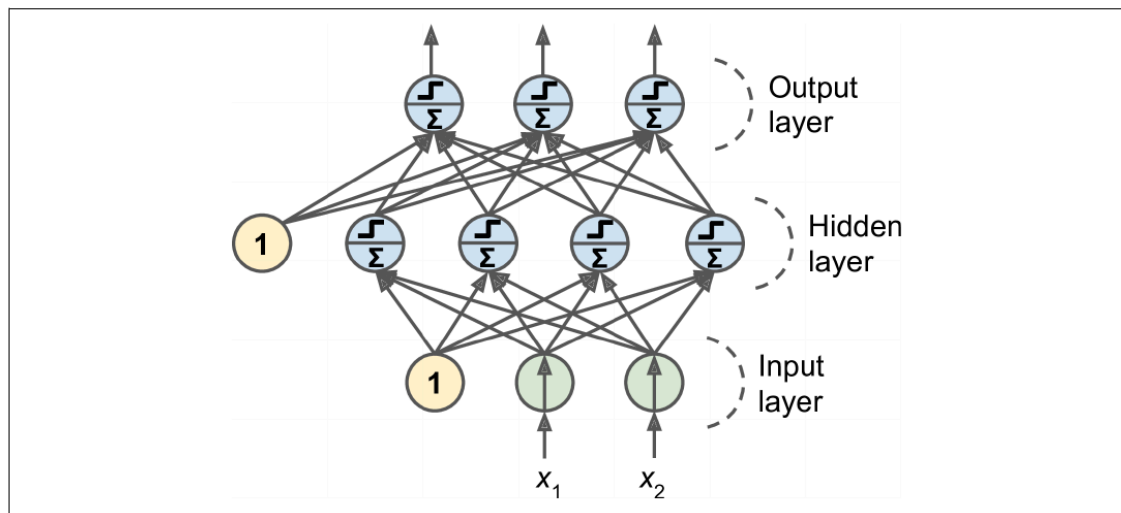
$$\frac{\partial J}{\partial \mathbf{w}^{(2)}} = \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{w}^{(2)}} \cdot \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial J}{\partial \mathbf{a}^{(3)}}$$

donde las últimas dos derivadas fueron también calculadas para $\frac{\partial J}{\partial \mathbf{w}^{(3)}}$. Eso quiere decir que la derivadas ya calculadas pueden ser reutilizadas o **propagadas hacia atrás** en el cálculo de Gradient Descent para otros pesos.

Redes multicapas y Deep Learning

- Es posible conectar múltiples neuronas entre sí tomando la salida de una neurona como la entrada para una o múltiples neuronas adicionales.
- Las conexiones que se producen entre neuronas conforman lo que se llama como **red neuronal**.
- Dependiendo de cuales sean las conexiones entre distintas neuronas, se habla de la **arquitectura** de la red neuronal.
- Las redes neuronales se dividen en **capas**, las cuales se dividen en las siguientes categorías:
 - Capas de entrada (input layers)
 - Capas de salida (output layers)
 - Capas ocultas/intermedias (hidden layers)

- Las redes multicapas están compuestas por una capa de entrada, una o varias capas ocultas y una capa de salida
- Cuando una red multicapa contiene muchas capas ocultas, hablamos de una **Red Neuronal Profunda** (Deep Neural Network - DNN)



- Al igual que en el perceptron, deseamos calcular los pesos w de todas las neuronas tales que la salida esté lo más cercana a los valores esperados.
- ¿Cómo podemos calcular tantos pesos a la vez?
 - **R:** Backpropagation

Backpropagation

- Back propagation es el procedimiento que se utiliza para calcular los valores de las derivadas parciales $\nabla_{w_j} J(\mathbf{w})$ de la función de costo sobre cada peso de la red.
- Se basa en la regla de la cadena para propagar los gradientes de capas posteriores para calcular los gradientes de capas anteriores.

Funciones de activación

- Las funciones de activación permiten agregar no linealidades a una neurona.
- Existen muchas funciones de activación. Algunas enfocadas en **regresión** (predicción de valores numéricos) y otras en **clasificación** (predicción de clases).
- Ejemplos:

Linear

$$\text{Linear}(x) = x$$

Sigmoide

- Útil en problemas de clasificación (entrega valores entre 0 y 1 que se interpretan como probabilidad)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

ReLU (Rectified Linear Unit)

- Utiliza en capas intermedias por su fácil cálculo de derivadas y agregar no linealidades de manera sencilla.

$$\text{ReLU}(x) = \max(0, x)$$

Leaky ReLU

- Uso similar a ReLU; evita que neuronas tenga activación cero.

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{si } x > 0 \\ \alpha x, & \text{si } x \leq 0 \end{cases}$$

Donde α es un valor pequeño y positivo.

Función Tangente Hiperbólica (Tanh)

- Se utiliza para introducir no linealidad en capas intermedias

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Función Softmax

- Se utiliza en problemas de clasificación multi clase (equivalente a sigmoide pero para varias clases).

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Google Colaboratory



- Proyecto basado en Jupyter Notebook (sistema ampliamente usado en Data Science)
- Permite escribir y ejecutar código Python en el navegador
 - Sin configuración requerida
 - Acceso gratuito a GPU (Graphic Process Unit)
 - Facilidad para compartir (en la nube)

02_08_How to Visualize Decision Trees using Matplotlib.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text

How do you understand how a decision tree makes predictions? One of the strengths of decision trees are that they are relatively easy to interpret as you can make a visualization based on your model. This is not only a powerful way to understand your model, but also to communicate how your model works to stakeholders.

Import Libraries

```
%matplotlib inline

import matplotlib.pyplot as plt
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

from sklearn import tree
```

Load the Dataset

The Iris dataset is one of datasets scikit-learn comes with that do not require the downloading of any file from some external website. The code below loads the iris dataset.

```
[2] data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0

Bibliotecas de Deep Learning

- Actualmente existe una amplia gama de bibliotecas para Deep Learning.
- Por su facilidad de uso y popularidad, utilizaremos la API de **Keras**, la cual viene incluida dentro de la biblioteca **Tensorflow**.
- Para poder utilizar Keras, es necesario tener previamente instalado Tensorflow (`pip install tensorflow`)



Creando redes neuronales en Keras

Para crear una red neuronal, debemos

- Definir la arquitectura de la red (modelo y capas)
- Compilar el modelo (selección de optimizador, función de pérdida y métricas)
- Entrenar modelo
- Usar!

```
In [13]: import tensorflow as tf  
print(tf.__version__)
```

```
2023-11-30 18:43:00.324236: I tensorflow/core/util/port.cc:111] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2023-11-30 18:43:00.352774: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
2023-11-30 18:43:00.352794: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
2023-11-30 18:43:00.352815: E tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
2023-11-30 18:43:00.358948: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2.14.1
```

```
In [14]: # Ejemplo usando fashion mnist
(X_train_full, y_train_full), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"] # 10 classes

X_train_full = X_train_full/255.0 # Escalamiento
```

```
In [15]: # Esta celda muestra algunas imágenes del dataset como ejemplo
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(15,15))

for i in range(36):
    ax = fig.add_subplot(6,6,i+1)
    ax.imshow(X_train_full[i], cmap='gist_yarg')
    ax.set_title(class_names[y_train_full[i]])

plt.show()
```



```
In [16]: # Acá armaremos nuestra red neuronal (la arquitectura)
# Recuerda que en este punto se debe escoger tanto el modelo, la cantidad de
# cantidad de neuronas, etc.
# Armado de nuestra red neuronal
model = tf.keras.models.Sequential() # Modelo secuencial
model.add(tf.keras.layers.Flatten(input_shape=[28, 28])) # Capa de entrada,
model.add(tf.keras.layers.Dense(300, activation='relu')) # Primera capa oculta
model.add(tf.keras.layers.Dense(100, activation='relu')) # Segunda capa oculta
model.add(tf.keras.layers.Dense(10, activation='softmax')) # Capa de salida,
# Importante
# -Para un modelo de clasificación multiclase debes tener tantas neuronas en
# tu dataset
# - La función de activación DEBE ser softmax de la capa de salida
```


Epoch 1/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.7463 - accuracy: 0.7545 - val_loss: 0.5341 - val_accuracy: 0.8084
Epoch 2/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.5045 - accuracy: 0.8251 - val_loss: 0.4652 - val_accuracy: 0.8374
Epoch 3/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.4577 - accuracy: 0.8398 - val_loss: 0.4307 - val_accuracy: 0.8474
Epoch 4/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.4297 - accuracy: 0.8500 - val_loss: 0.4179 - val_accuracy: 0.8558
Epoch 5/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.4084 - accuracy: 0.8570 - val_loss: 0.4166 - val_accuracy: 0.8526
Epoch 6/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3926 - accuracy: 0.8603 - val_loss: 0.3788 - val_accuracy: 0.8708
Epoch 7/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3768 - accuracy: 0.8657 - val_loss: 0.3852 - val_accuracy: 0.8627
Epoch 8/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3652 - accuracy: 0.8699 - val_loss: 0.3718 - val_accuracy: 0.8702
Epoch 9/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3540 - accuracy: 0.8740 - val_loss: 0.3716 - val_accuracy: 0.8688
Epoch 10/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3436 - accuracy: 0.8770 - val_loss: 0.3897 - val_accuracy: 0.8664
Epoch 11/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3345 - accuracy: 0.8811 - val_loss: 0.3636 - val_accuracy: 0.8767
Epoch 12/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3266 - accuracy: 0.8827 - val_loss: 0.3405 - val_accuracy: 0.8845
Epoch 13/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3195 - accuracy: 0.8850 - val_loss: 0.3322 - val_accuracy: 0.8854
Epoch 14/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3123 - accuracy: 0.8866 - val_loss: 0.3336 - val_accuracy: 0.8847
Epoch 15/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.3042 - accuracy: 0.8909 - val_loss: 0.3357 - val_accuracy: 0.8822
Epoch 16/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2984 - accuracy: 0.8924 - val_loss: 0.3403 - val_accuracy: 0.8778
Epoch 17/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2928 - accuracy: 0.8948 - val_loss: 0.3264 - val_accuracy: 0.8836
Epoch 18/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2869 - accuracy: 0.8958 - val_loss: 0.3253 - val_accuracy: 0.8842
Epoch 19/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2812 - ac

```

curacy: 0.8989 - val_loss: 0.3195 - val_accuracy: 0.8862
Epoch 20/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2761 - ac
curacy: 0.9004 - val_loss: 0.3257 - val_accuracy: 0.8848
Epoch 21/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2710 - ac
curacy: 0.9019 - val_loss: 0.3119 - val_accuracy: 0.8892
Epoch 22/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2656 - ac
curacy: 0.9031 - val_loss: 0.3111 - val_accuracy: 0.8912
Epoch 23/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2617 - ac
curacy: 0.9047 - val_loss: 0.3130 - val_accuracy: 0.8895
Epoch 24/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2563 - ac
curacy: 0.9076 - val_loss: 0.3265 - val_accuracy: 0.8830
Epoch 25/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2533 - ac
curacy: 0.9081 - val_loss: 0.3272 - val_accuracy: 0.8801
Epoch 26/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2478 - ac
curacy: 0.9108 - val_loss: 0.3109 - val_accuracy: 0.8891
Epoch 27/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2439 - ac
curacy: 0.9111 - val_loss: 0.3178 - val_accuracy: 0.8882
Epoch 28/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2398 - ac
curacy: 0.9129 - val_loss: 0.3185 - val_accuracy: 0.8878
Epoch 29/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2355 - ac
curacy: 0.9150 - val_loss: 0.3101 - val_accuracy: 0.8875
Epoch 30/30
1500/1500 [=====] - 2s 1ms/step - loss: 0.2328 - ac
curacy: 0.9143 - val_loss: 0.3113 - val_accuracy: 0.8916

```

```

In [21]: X_test = X_test/255.0 # nuevamente, escalamiento de set de datos de entrenan
loss, acc = model.evaluate(X_test, y_test, verbose=2)
print("Loss:", loss) # Función de pérdida
print("Accuracy:", acc)

```

```

313/313 - 0s - loss: 0.3461 - accuracy: 0.8780 - 257ms/epoch - 822us/step
Loss: 0.3461460769176483
Accuracy: 0.878000020980835

```

```

In [22]: import numpy as np
#dato = 56
#dato = 74
dato=12
prediction = model.predict(X_test)
print(prediction[12])
classes_x=np.argmax(prediction,axis=1)
print("Predicción:", class_names[classes_x[dato]])
print("Clase Real:", class_names[y_test[dato]])

```

```
313/313 [=====] - 0s 663us/step
[8.11329010e-05 1.07122504e-03 2.06618352e-04 3.89063731e-03
 3.26435023e-04 1.12326115e-01 8.56319966e-05 5.28286397e-01
 3.53616118e-01 1.09734072e-04]
Predicción: Sneaker
Clase Real: Sneaker
```

Regularización en NN

- La gran cantidad de parámetros que posee una red neuronal, si bien, entrega una gran flexibilidad de ajuste a una enorme cantidad de datos distintos, esto también las hace propensas al overfitting (sobre ajuste).
- Para evitar el overfitting, al igual en otros métodos, necesitamos utilizar técnicas de **regularización**.
- Algunas de las técnicas más populares son:
 - Regularización l_1 y l_2
 - Dropout (dilución)
 - Early stop (detención anticipada)

```
In [23]: # Ejemplo
# Imports
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
In [24]: df = pd.read_csv('data/nba.csv', index_col = 'Name')
df.head()
```


Out[24]:

	GP	MIN	PTS	FGM	FGA	FG%	3P Made	3PA	3P%	FTM	FTA	FT%	OREB
Name													
Brandon Ingram	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	25.0	1.6	2.3	69.9	0.7
Andrew Harrison	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	23.5	2.6	3.4	76.5	0.5
JaKarr Sampson	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	24.4	0.9	1.3	67.0	0.5
Malik Sealy	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	22.6	0.9	1.3	68.9	1.0
Matt Geiger	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	0.0	1.3	1.9	67.4	1.0

```
In [25]: # Botar faltantes
df.dropna(inplace = True)
# Guardar X datos
X = df.drop(columns = 'TARGET_5Yrs')
# Codificar nuestro objetivo
y = df['TARGET_5Yrs']
```

```
In [26]: # División entrenamiento, validación, prueba
X_train_full, X_test, y_train_full, y_test = train_test_split(X, y, test_size=0.2)
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full,
```

```
In [27]: # Escalamiento
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# Input shape
input_shape = X_train.shape[1]
print(input_shape)
```

19

```
In [28]: # Modelo sin regularización
model = Sequential()
# Primera capa oculta
model.add(Dense(19,
                input_dim = input_shape,
                activation = 'relu'))
model.add(Dense(10,
                activation = 'relu'))
model.add(Dense(1, activation = 'sigmoid'))

model.compile(loss = 'bce', optimizer = 'sgd', metrics=['accuracy']) # bce:
```

```
history = model.fit(X_train, y_train,  
                    validation_data = (X_val, y_val),  
                    epochs=100)
```

Epoch 1/100
27/27 [=====] - 0s 4ms/step - loss: 0.6729 - accuracy: 0.5812 - val_loss: 0.6443 - val_accuracy: 0.5915
Epoch 2/100
27/27 [=====] - 0s 1ms/step - loss: 0.6495 - accuracy: 0.6094 - val_loss: 0.6282 - val_accuracy: 0.6291
Epoch 3/100
27/27 [=====] - 0s 1ms/step - loss: 0.6362 - accuracy: 0.6329 - val_loss: 0.6174 - val_accuracy: 0.6385
Epoch 4/100
27/27 [=====] - 0s 1ms/step - loss: 0.6278 - accuracy: 0.6471 - val_loss: 0.6103 - val_accuracy: 0.6479
Epoch 5/100
27/27 [=====] - 0s 1ms/step - loss: 0.6220 - accuracy: 0.6647 - val_loss: 0.6052 - val_accuracy: 0.6432
Epoch 6/100
27/27 [=====] - 0s 1ms/step - loss: 0.6177 - accuracy: 0.6600 - val_loss: 0.6013 - val_accuracy: 0.6526
Epoch 7/100
27/27 [=====] - 0s 1ms/step - loss: 0.6143 - accuracy: 0.6647 - val_loss: 0.5984 - val_accuracy: 0.6526
Epoch 8/100
27/27 [=====] - 0s 1ms/step - loss: 0.6115 - accuracy: 0.6647 - val_loss: 0.5959 - val_accuracy: 0.6526
Epoch 9/100
27/27 [=====] - 0s 1ms/step - loss: 0.6091 - accuracy: 0.6671 - val_loss: 0.5936 - val_accuracy: 0.6620
Epoch 10/100
27/27 [=====] - 0s 1ms/step - loss: 0.6068 - accuracy: 0.6718 - val_loss: 0.5919 - val_accuracy: 0.6714
Epoch 11/100
27/27 [=====] - 0s 1ms/step - loss: 0.6049 - accuracy: 0.6706 - val_loss: 0.5904 - val_accuracy: 0.6714
Epoch 12/100
27/27 [=====] - 0s 1ms/step - loss: 0.6031 - accuracy: 0.6729 - val_loss: 0.5892 - val_accuracy: 0.6667
Epoch 13/100
27/27 [=====] - 0s 1ms/step - loss: 0.6014 - accuracy: 0.6741 - val_loss: 0.5879 - val_accuracy: 0.6761
Epoch 14/100
27/27 [=====] - 0s 1ms/step - loss: 0.5999 - accuracy: 0.6741 - val_loss: 0.5869 - val_accuracy: 0.6808
Epoch 15/100
27/27 [=====] - 0s 1ms/step - loss: 0.5985 - accuracy: 0.6753 - val_loss: 0.5861 - val_accuracy: 0.6808
Epoch 16/100
27/27 [=====] - 0s 1ms/step - loss: 0.5971 - accuracy: 0.6776 - val_loss: 0.5855 - val_accuracy: 0.6808
Epoch 17/100
27/27 [=====] - 0s 1ms/step - loss: 0.5959 - accuracy: 0.6835 - val_loss: 0.5850 - val_accuracy: 0.6808
Epoch 18/100
27/27 [=====] - 0s 1ms/step - loss: 0.5948 - accuracy: 0.6847 - val_loss: 0.5846 - val_accuracy: 0.6808
Epoch 19/100
27/27 [=====] - 0s 1ms/step - loss: 0.5938 - accuracy:

cy: 0.6847 - val_loss: 0.5841 - val_accuracy: 0.6854
Epoch 20/100
27/27 [=====] - 0s 1ms/step - loss: 0.5928 - accuracy: 0.6859 - val_loss: 0.5837 - val_accuracy: 0.6854
Epoch 21/100
27/27 [=====] - 0s 1ms/step - loss: 0.5919 - accuracy: 0.6859 - val_loss: 0.5834 - val_accuracy: 0.6854
Epoch 22/100
27/27 [=====] - 0s 1ms/step - loss: 0.5911 - accuracy: 0.6871 - val_loss: 0.5831 - val_accuracy: 0.6854
Epoch 23/100
27/27 [=====] - 0s 1ms/step - loss: 0.5902 - accuracy: 0.6882 - val_loss: 0.5830 - val_accuracy: 0.6854
Epoch 24/100
27/27 [=====] - 0s 1ms/step - loss: 0.5894 - accuracy: 0.6906 - val_loss: 0.5828 - val_accuracy: 0.6808
Epoch 25/100
27/27 [=====] - 0s 1ms/step - loss: 0.5886 - accuracy: 0.6894 - val_loss: 0.5827 - val_accuracy: 0.6808
Epoch 26/100
27/27 [=====] - 0s 1ms/step - loss: 0.5877 - accuracy: 0.6918 - val_loss: 0.5825 - val_accuracy: 0.6761
Epoch 27/100
27/27 [=====] - 0s 1ms/step - loss: 0.5870 - accuracy: 0.6929 - val_loss: 0.5823 - val_accuracy: 0.6761
Epoch 28/100
27/27 [=====] - 0s 1ms/step - loss: 0.5863 - accuracy: 0.6906 - val_loss: 0.5823 - val_accuracy: 0.6761
Epoch 29/100
27/27 [=====] - 0s 1ms/step - loss: 0.5857 - accuracy: 0.6929 - val_loss: 0.5822 - val_accuracy: 0.6761
Epoch 30/100
27/27 [=====] - 0s 1ms/step - loss: 0.5849 - accuracy: 0.6941 - val_loss: 0.5820 - val_accuracy: 0.6761
Epoch 31/100
27/27 [=====] - 0s 1ms/step - loss: 0.5843 - accuracy: 0.6965 - val_loss: 0.5818 - val_accuracy: 0.6761
Epoch 32/100
27/27 [=====] - 0s 1ms/step - loss: 0.5836 - accuracy: 0.6976 - val_loss: 0.5815 - val_accuracy: 0.6808
Epoch 33/100
27/27 [=====] - 0s 1ms/step - loss: 0.5830 - accuracy: 0.6988 - val_loss: 0.5813 - val_accuracy: 0.6808
Epoch 34/100
27/27 [=====] - 0s 1ms/step - loss: 0.5824 - accuracy: 0.7012 - val_loss: 0.5813 - val_accuracy: 0.6808
Epoch 35/100
27/27 [=====] - 0s 1ms/step - loss: 0.5819 - accuracy: 0.7000 - val_loss: 0.5811 - val_accuracy: 0.6808
Epoch 36/100
27/27 [=====] - 0s 1ms/step - loss: 0.5812 - accuracy: 0.7047 - val_loss: 0.5811 - val_accuracy: 0.6761
Epoch 37/100
27/27 [=====] - 0s 1ms/step - loss: 0.5806 - accuracy: 0.7047 - val_loss: 0.5810 - val_accuracy: 0.6761
Epoch 38/100

27/27 [=====] - 0s 1ms/step - loss: 0.5800 - accuracy: 0.7047 - val_loss: 0.5809 - val_accuracy: 0.6761
Epoch 39/100
27/27 [=====] - 0s 1ms/step - loss: 0.5796 - accuracy: 0.7047 - val_loss: 0.5808 - val_accuracy: 0.6667
Epoch 40/100
27/27 [=====] - 0s 1ms/step - loss: 0.5791 - accuracy: 0.7035 - val_loss: 0.5810 - val_accuracy: 0.6714
Epoch 41/100
27/27 [=====] - 0s 1ms/step - loss: 0.5785 - accuracy: 0.7047 - val_loss: 0.5810 - val_accuracy: 0.6714
Epoch 42/100
27/27 [=====] - 0s 1ms/step - loss: 0.5780 - accuracy: 0.7035 - val_loss: 0.5809 - val_accuracy: 0.6667
Epoch 43/100
27/27 [=====] - 0s 1ms/step - loss: 0.5774 - accuracy: 0.7035 - val_loss: 0.5809 - val_accuracy: 0.6667
Epoch 44/100
27/27 [=====] - 0s 1ms/step - loss: 0.5770 - accuracy: 0.7047 - val_loss: 0.5809 - val_accuracy: 0.6667
Epoch 45/100
27/27 [=====] - 0s 1ms/step - loss: 0.5765 - accuracy: 0.7024 - val_loss: 0.5809 - val_accuracy: 0.6714
Epoch 46/100
27/27 [=====] - 0s 1ms/step - loss: 0.5761 - accuracy: 0.7059 - val_loss: 0.5808 - val_accuracy: 0.6714
Epoch 47/100
27/27 [=====] - 0s 1ms/step - loss: 0.5756 - accuracy: 0.7024 - val_loss: 0.5808 - val_accuracy: 0.6761
Epoch 48/100
27/27 [=====] - 0s 1ms/step - loss: 0.5751 - accuracy: 0.7047 - val_loss: 0.5810 - val_accuracy: 0.6714
Epoch 49/100
27/27 [=====] - 0s 1ms/step - loss: 0.5748 - accuracy: 0.7024 - val_loss: 0.5810 - val_accuracy: 0.6714
Epoch 50/100
27/27 [=====] - 0s 1ms/step - loss: 0.5743 - accuracy: 0.7035 - val_loss: 0.5810 - val_accuracy: 0.6714
Epoch 51/100
27/27 [=====] - 0s 1ms/step - loss: 0.5740 - accuracy: 0.7035 - val_loss: 0.5811 - val_accuracy: 0.6714
Epoch 52/100
27/27 [=====] - 0s 1ms/step - loss: 0.5735 - accuracy: 0.7035 - val_loss: 0.5812 - val_accuracy: 0.6714
Epoch 53/100
27/27 [=====] - 0s 1ms/step - loss: 0.5731 - accuracy: 0.7024 - val_loss: 0.5809 - val_accuracy: 0.6761
Epoch 54/100
27/27 [=====] - 0s 1ms/step - loss: 0.5728 - accuracy: 0.7047 - val_loss: 0.5810 - val_accuracy: 0.6761
Epoch 55/100
27/27 [=====] - 0s 1ms/step - loss: 0.5723 - accuracy: 0.7047 - val_loss: 0.5808 - val_accuracy: 0.6761
Epoch 56/100
27/27 [=====] - 0s 1ms/step - loss: 0.5721 - accuracy: 0.7047 - val_loss: 0.5808 - val_accuracy: 0.6761

Epoch 57/100
27/27 [=====] - 0s 1ms/step - loss: 0.5716 - accuracy: 0.7059 - val_loss: 0.5808 - val_accuracy: 0.6761
Epoch 58/100
27/27 [=====] - 0s 1ms/step - loss: 0.5713 - accuracy: 0.7047 - val_loss: 0.5809 - val_accuracy: 0.6714
Epoch 59/100
27/27 [=====] - 0s 1ms/step - loss: 0.5710 - accuracy: 0.7082 - val_loss: 0.5809 - val_accuracy: 0.6714
Epoch 60/100
27/27 [=====] - 0s 2ms/step - loss: 0.5705 - accuracy: 0.7059 - val_loss: 0.5809 - val_accuracy: 0.6714
Epoch 61/100
27/27 [=====] - 0s 1ms/step - loss: 0.5701 - accuracy: 0.7094 - val_loss: 0.5808 - val_accuracy: 0.6761
Epoch 62/100
27/27 [=====] - 0s 1ms/step - loss: 0.5698 - accuracy: 0.7106 - val_loss: 0.5810 - val_accuracy: 0.6714
Epoch 63/100
27/27 [=====] - 0s 1ms/step - loss: 0.5696 - accuracy: 0.7082 - val_loss: 0.5808 - val_accuracy: 0.6808
Epoch 64/100
27/27 [=====] - 0s 1ms/step - loss: 0.5692 - accuracy: 0.7094 - val_loss: 0.5806 - val_accuracy: 0.6808
Epoch 65/100
27/27 [=====] - 0s 1ms/step - loss: 0.5688 - accuracy: 0.7094 - val_loss: 0.5808 - val_accuracy: 0.6808
Epoch 66/100
27/27 [=====] - 0s 1ms/step - loss: 0.5685 - accuracy: 0.7094 - val_loss: 0.5809 - val_accuracy: 0.6808
Epoch 67/100
27/27 [=====] - 0s 1ms/step - loss: 0.5683 - accuracy: 0.7082 - val_loss: 0.5808 - val_accuracy: 0.6808
Epoch 68/100
27/27 [=====] - 0s 1ms/step - loss: 0.5679 - accuracy: 0.7071 - val_loss: 0.5809 - val_accuracy: 0.6808
Epoch 69/100
27/27 [=====] - 0s 1ms/step - loss: 0.5675 - accuracy: 0.7094 - val_loss: 0.5809 - val_accuracy: 0.6808
Epoch 70/100
27/27 [=====] - 0s 1ms/step - loss: 0.5673 - accuracy: 0.7082 - val_loss: 0.5810 - val_accuracy: 0.6808
Epoch 71/100
27/27 [=====] - 0s 1ms/step - loss: 0.5669 - accuracy: 0.7106 - val_loss: 0.5811 - val_accuracy: 0.6761
Epoch 72/100
27/27 [=====] - 0s 1ms/step - loss: 0.5668 - accuracy: 0.7094 - val_loss: 0.5811 - val_accuracy: 0.6761
Epoch 73/100
27/27 [=====] - 0s 1ms/step - loss: 0.5664 - accuracy: 0.7106 - val_loss: 0.5812 - val_accuracy: 0.6761
Epoch 74/100
27/27 [=====] - 0s 1ms/step - loss: 0.5661 - accuracy: 0.7082 - val_loss: 0.5812 - val_accuracy: 0.6761
Epoch 75/100
27/27 [=====] - 0s 1ms/step - loss: 0.5658 - accuracy:

cy: 0.7094 - val_loss: 0.5813 - val_accuracy: 0.6761
Epoch 76/100
27/27 [=====] - 0s 1ms/step - loss: 0.5656 - accuracy: 0.7094 - val_loss: 0.5810 - val_accuracy: 0.6854
Epoch 77/100
27/27 [=====] - 0s 1ms/step - loss: 0.5654 - accuracy: 0.7094 - val_loss: 0.5811 - val_accuracy: 0.6854
Epoch 78/100
27/27 [=====] - 0s 1ms/step - loss: 0.5650 - accuracy: 0.7094 - val_loss: 0.5811 - val_accuracy: 0.6808
Epoch 79/100
27/27 [=====] - 0s 1ms/step - loss: 0.5647 - accuracy: 0.7106 - val_loss: 0.5810 - val_accuracy: 0.6854
Epoch 80/100
27/27 [=====] - 0s 1ms/step - loss: 0.5645 - accuracy: 0.7106 - val_loss: 0.5810 - val_accuracy: 0.6854
Epoch 81/100
27/27 [=====] - 0s 1ms/step - loss: 0.5642 - accuracy: 0.7106 - val_loss: 0.5810 - val_accuracy: 0.6854
Epoch 82/100
27/27 [=====] - 0s 1ms/step - loss: 0.5639 - accuracy: 0.7118 - val_loss: 0.5811 - val_accuracy: 0.6854
Epoch 83/100
27/27 [=====] - 0s 1ms/step - loss: 0.5637 - accuracy: 0.7106 - val_loss: 0.5813 - val_accuracy: 0.6761
Epoch 84/100
27/27 [=====] - 0s 1ms/step - loss: 0.5634 - accuracy: 0.7106 - val_loss: 0.5812 - val_accuracy: 0.6808
Epoch 85/100
27/27 [=====] - 0s 1ms/step - loss: 0.5632 - accuracy: 0.7118 - val_loss: 0.5811 - val_accuracy: 0.6808
Epoch 86/100
27/27 [=====] - 0s 1ms/step - loss: 0.5630 - accuracy: 0.7118 - val_loss: 0.5812 - val_accuracy: 0.6808
Epoch 87/100
27/27 [=====] - 0s 1ms/step - loss: 0.5626 - accuracy: 0.7118 - val_loss: 0.5812 - val_accuracy: 0.6761
Epoch 88/100
27/27 [=====] - 0s 1ms/step - loss: 0.5624 - accuracy: 0.7118 - val_loss: 0.5811 - val_accuracy: 0.6808
Epoch 89/100
27/27 [=====] - 0s 1ms/step - loss: 0.5621 - accuracy: 0.7118 - val_loss: 0.5812 - val_accuracy: 0.6761
Epoch 90/100
27/27 [=====] - 0s 1ms/step - loss: 0.5618 - accuracy: 0.7106 - val_loss: 0.5812 - val_accuracy: 0.6761
Epoch 91/100
27/27 [=====] - 0s 1ms/step - loss: 0.5616 - accuracy: 0.7118 - val_loss: 0.5814 - val_accuracy: 0.6808
Epoch 92/100
27/27 [=====] - 0s 1ms/step - loss: 0.5614 - accuracy: 0.7129 - val_loss: 0.5816 - val_accuracy: 0.6808
Epoch 93/100
27/27 [=====] - 0s 1ms/step - loss: 0.5612 - accuracy: 0.7129 - val_loss: 0.5815 - val_accuracy: 0.6808
Epoch 94/100

```

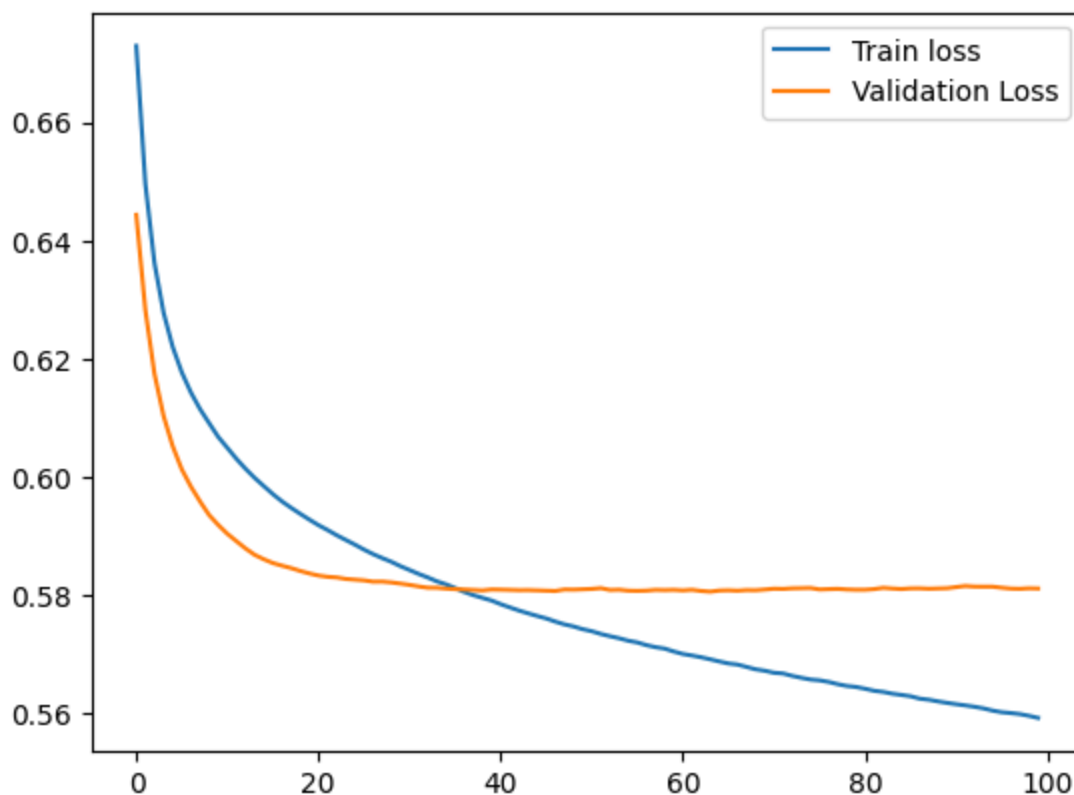
27/27 [=====] - 0s 1ms/step - loss: 0.5609 - accuracy: 0.7118 - val_loss: 0.5815 - val_accuracy: 0.6761
Epoch 95/100
27/27 [=====] - 0s 1ms/step - loss: 0.5605 - accuracy: 0.7141 - val_loss: 0.5815 - val_accuracy: 0.6854
Epoch 96/100
27/27 [=====] - 0s 1ms/step - loss: 0.5603 - accuracy: 0.7129 - val_loss: 0.5813 - val_accuracy: 0.6948
Epoch 97/100
27/27 [=====] - 0s 1ms/step - loss: 0.5601 - accuracy: 0.7153 - val_loss: 0.5812 - val_accuracy: 0.6854
Epoch 98/100
27/27 [=====] - 0s 1ms/step - loss: 0.5599 - accuracy: 0.7176 - val_loss: 0.5811 - val_accuracy: 0.6854
Epoch 99/100
27/27 [=====] - 0s 1ms/step - loss: 0.5596 - accuracy: 0.7176 - val_loss: 0.5812 - val_accuracy: 0.6854
Epoch 100/100
27/27 [=====] - 0s 1ms/step - loss: 0.5593 - accuracy: 0.7153 - val_loss: 0.5812 - val_accuracy: 0.6854

```

```

In [29]: # Visualizar la perdida
plt.plot(history.history['loss'], label='Train loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend();

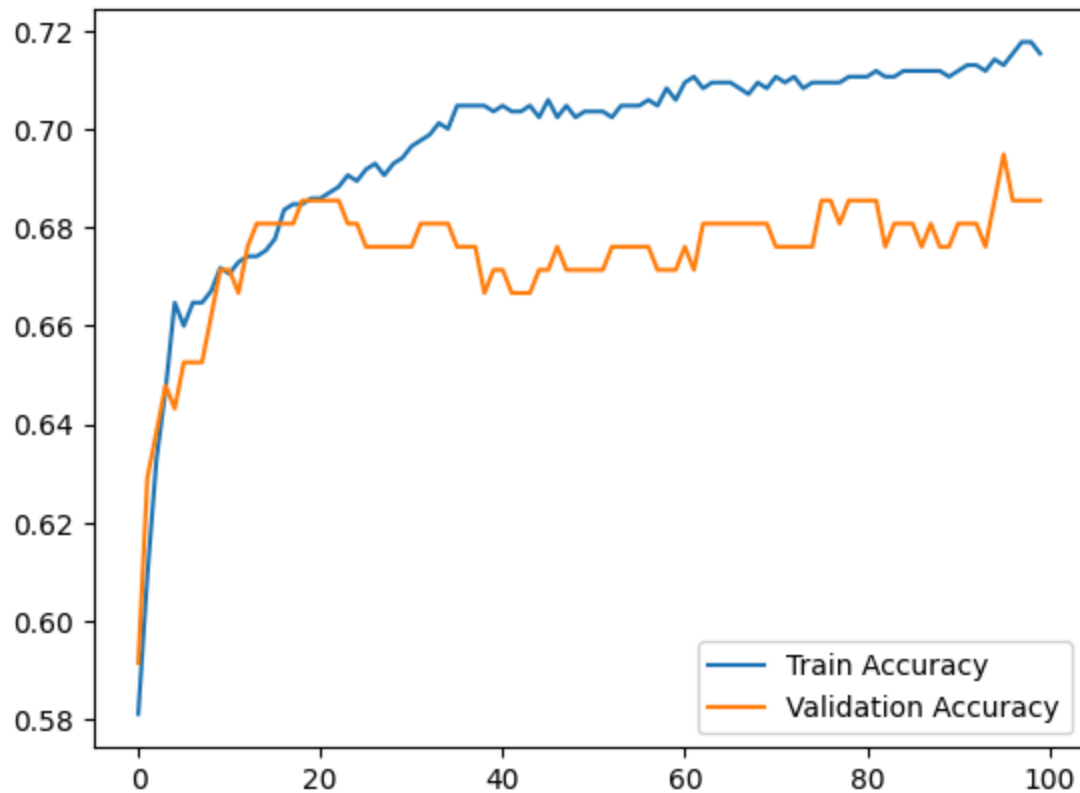
```



```

In [30]: # Visualizar el accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend();

```

Regularización l_1 y l_2

- Las regularización l_1 y l_2 agregan factores a la función de costo de una red neuronal para impedir que los pesos de esta tomen cualquier valor (multiplicadores de Lagrange).
- Las regularizaciones de tipo l_1 y l_2 se definen de la siguiente manera sobre la función de costo:
 - Regularización l_1

$$J = \frac{1}{m} \sum_{i=1}^m \left(\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)} \right)^2 + \alpha \frac{1}{n} \sum_{j=1}^n |\mathbf{w}_j|$$

- Regularización l_2

$$J = \frac{1}{m} \sum_{i=1}^m \left(\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)} \right)^2 + \alpha \frac{1}{2n} \sum_{j=1}^n \mathbf{w}_j^2$$

```
In [31]: from tensorflow.keras.regularizers import l2
# Con Regularización l2
# Modelo secuencial
model = Sequential()
# Primera capa oculta
```

```
model.add(Dense(19, # Cuántas neuronas tienes en tu primera capa oculta
                input_dim = input_shape, # ¿Cuál es la forma de tus caracter
                activation = 'relu',
                kernel_regularizer=l2(0.01))) # ¿Qué función de activación e
model.add(Dense(10,
                activation = 'relu',
                kernel_regularizer=l2(0.01)))
model.add(Dense(1, activation = 'sigmoid'))
model.compile(loss = 'bce', optimizer = 'adam', metrics=['accuracy'])
history_l2 = model.fit(X_train, y_train,
                      validation_data = (X_val, y_val),
                      epochs=100)
```

Epoch 1/100
27/27 [=====] - 0s 4ms/step - loss: 0.9664 - accuracy: 0.6247 - val_loss: 0.9254 - val_accuracy: 0.6526
Epoch 2/100
27/27 [=====] - 0s 1ms/step - loss: 0.9088 - accuracy: 0.6718 - val_loss: 0.8847 - val_accuracy: 0.6714
Epoch 3/100
27/27 [=====] - 0s 1ms/step - loss: 0.8738 - accuracy: 0.6788 - val_loss: 0.8524 - val_accuracy: 0.6901
Epoch 4/100
27/27 [=====] - 0s 1ms/step - loss: 0.8440 - accuracy: 0.6859 - val_loss: 0.8274 - val_accuracy: 0.6948
Epoch 5/100
27/27 [=====] - 0s 1ms/step - loss: 0.8173 - accuracy: 0.6918 - val_loss: 0.8060 - val_accuracy: 0.6995
Epoch 6/100
27/27 [=====] - 0s 1ms/step - loss: 0.7949 - accuracy: 0.6882 - val_loss: 0.7861 - val_accuracy: 0.6714
Epoch 7/100
27/27 [=====] - 0s 1ms/step - loss: 0.7730 - accuracy: 0.6988 - val_loss: 0.7682 - val_accuracy: 0.6995
Epoch 8/100
27/27 [=====] - 0s 1ms/step - loss: 0.7539 - accuracy: 0.7035 - val_loss: 0.7519 - val_accuracy: 0.6901
Epoch 9/100
27/27 [=====] - 0s 1ms/step - loss: 0.7382 - accuracy: 0.7012 - val_loss: 0.7370 - val_accuracy: 0.6995
Epoch 10/100
27/27 [=====] - 0s 1ms/step - loss: 0.7222 - accuracy: 0.7071 - val_loss: 0.7262 - val_accuracy: 0.7042
Epoch 11/100
27/27 [=====] - 0s 1ms/step - loss: 0.7086 - accuracy: 0.7071 - val_loss: 0.7141 - val_accuracy: 0.6995
Epoch 12/100
27/27 [=====] - 0s 1ms/step - loss: 0.6967 - accuracy: 0.7035 - val_loss: 0.7058 - val_accuracy: 0.6948
Epoch 13/100
27/27 [=====] - 0s 1ms/step - loss: 0.6865 - accuracy: 0.7047 - val_loss: 0.6965 - val_accuracy: 0.6948
Epoch 14/100
27/27 [=====] - 0s 1ms/step - loss: 0.6770 - accuracy: 0.7059 - val_loss: 0.6886 - val_accuracy: 0.6901
Epoch 15/100
27/27 [=====] - 0s 1ms/step - loss: 0.6689 - accuracy: 0.7071 - val_loss: 0.6830 - val_accuracy: 0.6948
Epoch 16/100
27/27 [=====] - 0s 1ms/step - loss: 0.6616 - accuracy: 0.7106 - val_loss: 0.6778 - val_accuracy: 0.6808
Epoch 17/100
27/27 [=====] - 0s 1ms/step - loss: 0.6543 - accuracy: 0.7141 - val_loss: 0.6747 - val_accuracy: 0.6854
Epoch 18/100
27/27 [=====] - 0s 1ms/step - loss: 0.6478 - accuracy: 0.7082 - val_loss: 0.6661 - val_accuracy: 0.6948
Epoch 19/100
27/27 [=====] - 0s 1ms/step - loss: 0.6430 - accuracy:

cy: 0.7129 - val_loss: 0.6619 - val_accuracy: 0.6901
Epoch 20/100
27/27 [=====] - 0s 1ms/step - loss: 0.6369 - accuracy: 0.7153 - val_loss: 0.6594 - val_accuracy: 0.6901
Epoch 21/100
27/27 [=====] - 0s 1ms/step - loss: 0.6318 - accuracy: 0.7153 - val_loss: 0.6537 - val_accuracy: 0.6948
Epoch 22/100
27/27 [=====] - 0s 1ms/step - loss: 0.6273 - accuracy: 0.7165 - val_loss: 0.6532 - val_accuracy: 0.6901
Epoch 23/100
27/27 [=====] - 0s 1ms/step - loss: 0.6237 - accuracy: 0.7247 - val_loss: 0.6473 - val_accuracy: 0.6948
Epoch 24/100
27/27 [=====] - 0s 1ms/step - loss: 0.6198 - accuracy: 0.7224 - val_loss: 0.6475 - val_accuracy: 0.6854
Epoch 25/100
27/27 [=====] - 0s 1ms/step - loss: 0.6165 - accuracy: 0.7212 - val_loss: 0.6435 - val_accuracy: 0.6901
Epoch 26/100
27/27 [=====] - 0s 1ms/step - loss: 0.6137 - accuracy: 0.7247 - val_loss: 0.6412 - val_accuracy: 0.6995
Epoch 27/100
27/27 [=====] - 0s 1ms/step - loss: 0.6111 - accuracy: 0.7235 - val_loss: 0.6391 - val_accuracy: 0.6948
Epoch 28/100
27/27 [=====] - 0s 1ms/step - loss: 0.6086 - accuracy: 0.7247 - val_loss: 0.6370 - val_accuracy: 0.6948
Epoch 29/100
27/27 [=====] - 0s 1ms/step - loss: 0.6061 - accuracy: 0.7212 - val_loss: 0.6379 - val_accuracy: 0.6901
Epoch 30/100
27/27 [=====] - 0s 1ms/step - loss: 0.6032 - accuracy: 0.7259 - val_loss: 0.6349 - val_accuracy: 0.6854
Epoch 31/100
27/27 [=====] - 0s 1ms/step - loss: 0.6015 - accuracy: 0.7259 - val_loss: 0.6326 - val_accuracy: 0.6901
Epoch 32/100
27/27 [=====] - 0s 1ms/step - loss: 0.6001 - accuracy: 0.7306 - val_loss: 0.6325 - val_accuracy: 0.7042
Epoch 33/100
27/27 [=====] - 0s 2ms/step - loss: 0.5981 - accuracy: 0.7188 - val_loss: 0.6311 - val_accuracy: 0.6901
Epoch 34/100
27/27 [=====] - 0s 1ms/step - loss: 0.5963 - accuracy: 0.7271 - val_loss: 0.6293 - val_accuracy: 0.7042
Epoch 35/100
27/27 [=====] - 0s 1ms/step - loss: 0.5935 - accuracy: 0.7294 - val_loss: 0.6311 - val_accuracy: 0.6761
Epoch 36/100
27/27 [=====] - 0s 1ms/step - loss: 0.5928 - accuracy: 0.7282 - val_loss: 0.6272 - val_accuracy: 0.6995
Epoch 37/100
27/27 [=====] - 0s 1ms/step - loss: 0.5920 - accuracy: 0.7282 - val_loss: 0.6247 - val_accuracy: 0.7042
Epoch 38/100

27/27 [=====] - 0s 1ms/step - loss: 0.5900 - accuracy: 0.7306 - val_loss: 0.6257 - val_accuracy: 0.7089
Epoch 39/100

27/27 [=====] - 0s 1ms/step - loss: 0.5886 - accuracy: 0.7271 - val_loss: 0.6291 - val_accuracy: 0.6761
Epoch 40/100

27/27 [=====] - 0s 1ms/step - loss: 0.5876 - accuracy: 0.7306 - val_loss: 0.6248 - val_accuracy: 0.7042
Epoch 41/100

27/27 [=====] - 0s 1ms/step - loss: 0.5854 - accuracy: 0.7282 - val_loss: 0.6246 - val_accuracy: 0.6948
Epoch 42/100

27/27 [=====] - 0s 1ms/step - loss: 0.5849 - accuracy: 0.7224 - val_loss: 0.6236 - val_accuracy: 0.6901
Epoch 43/100

27/27 [=====] - 0s 1ms/step - loss: 0.5824 - accuracy: 0.7282 - val_loss: 0.6255 - val_accuracy: 0.6808
Epoch 44/100

27/27 [=====] - 0s 1ms/step - loss: 0.5822 - accuracy: 0.7329 - val_loss: 0.6236 - val_accuracy: 0.6901
Epoch 45/100

27/27 [=====] - 0s 1ms/step - loss: 0.5816 - accuracy: 0.7365 - val_loss: 0.6221 - val_accuracy: 0.6995
Epoch 46/100

27/27 [=====] - 0s 2ms/step - loss: 0.5808 - accuracy: 0.7329 - val_loss: 0.6212 - val_accuracy: 0.6995
Epoch 47/100

27/27 [=====] - 0s 1ms/step - loss: 0.5803 - accuracy: 0.7329 - val_loss: 0.6208 - val_accuracy: 0.6995
Epoch 48/100

27/27 [=====] - 0s 2ms/step - loss: 0.5790 - accuracy: 0.7341 - val_loss: 0.6208 - val_accuracy: 0.6995
Epoch 49/100

27/27 [=====] - 0s 1ms/step - loss: 0.5776 - accuracy: 0.7306 - val_loss: 0.6211 - val_accuracy: 0.6854
Epoch 50/100

27/27 [=====] - 0s 1ms/step - loss: 0.5771 - accuracy: 0.7318 - val_loss: 0.6256 - val_accuracy: 0.6948
Epoch 51/100

27/27 [=====] - 0s 1ms/step - loss: 0.5763 - accuracy: 0.7376 - val_loss: 0.6212 - val_accuracy: 0.6808
Epoch 52/100

27/27 [=====] - 0s 1ms/step - loss: 0.5756 - accuracy: 0.7329 - val_loss: 0.6196 - val_accuracy: 0.6995
Epoch 53/100

27/27 [=====] - 0s 1ms/step - loss: 0.5740 - accuracy: 0.7318 - val_loss: 0.6260 - val_accuracy: 0.6808
Epoch 54/100

27/27 [=====] - 0s 1ms/step - loss: 0.5752 - accuracy: 0.7341 - val_loss: 0.6232 - val_accuracy: 0.6854
Epoch 55/100

27/27 [=====] - 0s 1ms/step - loss: 0.5749 - accuracy: 0.7353 - val_loss: 0.6205 - val_accuracy: 0.6948
Epoch 56/100

27/27 [=====] - 0s 1ms/step - loss: 0.5738 - accuracy: 0.7353 - val_loss: 0.6230 - val_accuracy: 0.6808

Epoch 57/100
27/27 [=====] - 0s 1ms/step - loss: 0.5745 - accuracy: 0.7353 - val_loss: 0.6248 - val_accuracy: 0.6714
Epoch 58/100
27/27 [=====] - 0s 1ms/step - loss: 0.5721 - accuracy: 0.7353 - val_loss: 0.6240 - val_accuracy: 0.6901
Epoch 59/100
27/27 [=====] - 0s 1ms/step - loss: 0.5708 - accuracy: 0.7365 - val_loss: 0.6224 - val_accuracy: 0.6901
Epoch 60/100
27/27 [=====] - 0s 1ms/step - loss: 0.5721 - accuracy: 0.7329 - val_loss: 0.6194 - val_accuracy: 0.6854
Epoch 61/100
27/27 [=====] - 0s 1ms/step - loss: 0.5728 - accuracy: 0.7341 - val_loss: 0.6249 - val_accuracy: 0.6761
Epoch 62/100
27/27 [=====] - 0s 1ms/step - loss: 0.5710 - accuracy: 0.7447 - val_loss: 0.6218 - val_accuracy: 0.6854
Epoch 63/100
27/27 [=====] - 0s 1ms/step - loss: 0.5711 - accuracy: 0.7318 - val_loss: 0.6263 - val_accuracy: 0.6808
Epoch 64/100
27/27 [=====] - 0s 1ms/step - loss: 0.5688 - accuracy: 0.7424 - val_loss: 0.6204 - val_accuracy: 0.6901
Epoch 65/100
27/27 [=====] - 0s 1ms/step - loss: 0.5701 - accuracy: 0.7329 - val_loss: 0.6289 - val_accuracy: 0.6761
Epoch 66/100
27/27 [=====] - 0s 1ms/step - loss: 0.5686 - accuracy: 0.7341 - val_loss: 0.6256 - val_accuracy: 0.6808
Epoch 67/100
27/27 [=====] - 0s 1ms/step - loss: 0.5666 - accuracy: 0.7412 - val_loss: 0.6259 - val_accuracy: 0.6667
Epoch 68/100
27/27 [=====] - 0s 1ms/step - loss: 0.5670 - accuracy: 0.7388 - val_loss: 0.6260 - val_accuracy: 0.6714
Epoch 69/100
27/27 [=====] - 0s 1ms/step - loss: 0.5662 - accuracy: 0.7365 - val_loss: 0.6261 - val_accuracy: 0.6667
Epoch 70/100
27/27 [=====] - 0s 1ms/step - loss: 0.5660 - accuracy: 0.7424 - val_loss: 0.6253 - val_accuracy: 0.6761
Epoch 71/100
27/27 [=====] - 0s 1ms/step - loss: 0.5670 - accuracy: 0.7388 - val_loss: 0.6262 - val_accuracy: 0.6761
Epoch 72/100
27/27 [=====] - 0s 1ms/step - loss: 0.5653 - accuracy: 0.7400 - val_loss: 0.6296 - val_accuracy: 0.6761
Epoch 73/100
27/27 [=====] - 0s 1ms/step - loss: 0.5664 - accuracy: 0.7447 - val_loss: 0.6258 - val_accuracy: 0.6714
Epoch 74/100
27/27 [=====] - 0s 1ms/step - loss: 0.5677 - accuracy: 0.7424 - val_loss: 0.6278 - val_accuracy: 0.6808
Epoch 75/100
27/27 [=====] - 0s 1ms/step - loss: 0.5648 - accuracy:

cy: 0.7435 - val_loss: 0.6269 - val_accuracy: 0.6761
Epoch 76/100
27/27 [=====] - 0s 1ms/step - loss: 0.5649 - accuracy: 0.7400 - val_loss: 0.6279 - val_accuracy: 0.6808
Epoch 77/100
27/27 [=====] - 0s 1ms/step - loss: 0.5647 - accuracy: 0.7365 - val_loss: 0.6298 - val_accuracy: 0.6667
Epoch 78/100
27/27 [=====] - 0s 1ms/step - loss: 0.5655 - accuracy: 0.7400 - val_loss: 0.6295 - val_accuracy: 0.6667
Epoch 79/100
27/27 [=====] - 0s 1ms/step - loss: 0.5637 - accuracy: 0.7400 - val_loss: 0.6247 - val_accuracy: 0.6761
Epoch 80/100
27/27 [=====] - 0s 1ms/step - loss: 0.5627 - accuracy: 0.7447 - val_loss: 0.6300 - val_accuracy: 0.6667
Epoch 81/100
27/27 [=====] - 0s 1ms/step - loss: 0.5635 - accuracy: 0.7435 - val_loss: 0.6341 - val_accuracy: 0.6808
Epoch 82/100
27/27 [=====] - 0s 1ms/step - loss: 0.5624 - accuracy: 0.7424 - val_loss: 0.6292 - val_accuracy: 0.6761
Epoch 83/100
27/27 [=====] - 0s 1ms/step - loss: 0.5612 - accuracy: 0.7471 - val_loss: 0.6319 - val_accuracy: 0.6667
Epoch 84/100
27/27 [=====] - 0s 1ms/step - loss: 0.5603 - accuracy: 0.7459 - val_loss: 0.6303 - val_accuracy: 0.6714
Epoch 85/100
27/27 [=====] - 0s 1ms/step - loss: 0.5605 - accuracy: 0.7471 - val_loss: 0.6280 - val_accuracy: 0.6714
Epoch 86/100
27/27 [=====] - 0s 1ms/step - loss: 0.5607 - accuracy: 0.7471 - val_loss: 0.6315 - val_accuracy: 0.6714
Epoch 87/100
27/27 [=====] - 0s 1ms/step - loss: 0.5626 - accuracy: 0.7412 - val_loss: 0.6289 - val_accuracy: 0.6667
Epoch 88/100
27/27 [=====] - 0s 1ms/step - loss: 0.5610 - accuracy: 0.7388 - val_loss: 0.6360 - val_accuracy: 0.6573
Epoch 89/100
27/27 [=====] - 0s 1ms/step - loss: 0.5598 - accuracy: 0.7435 - val_loss: 0.6287 - val_accuracy: 0.6573
Epoch 90/100
27/27 [=====] - 0s 1ms/step - loss: 0.5602 - accuracy: 0.7435 - val_loss: 0.6301 - val_accuracy: 0.6714
Epoch 91/100
27/27 [=====] - 0s 1ms/step - loss: 0.5600 - accuracy: 0.7447 - val_loss: 0.6330 - val_accuracy: 0.6714
Epoch 92/100
27/27 [=====] - 0s 1ms/step - loss: 0.5600 - accuracy: 0.7435 - val_loss: 0.6368 - val_accuracy: 0.6620
Epoch 93/100
27/27 [=====] - 0s 1ms/step - loss: 0.5600 - accuracy: 0.7412 - val_loss: 0.6350 - val_accuracy: 0.6573
Epoch 94/100

```

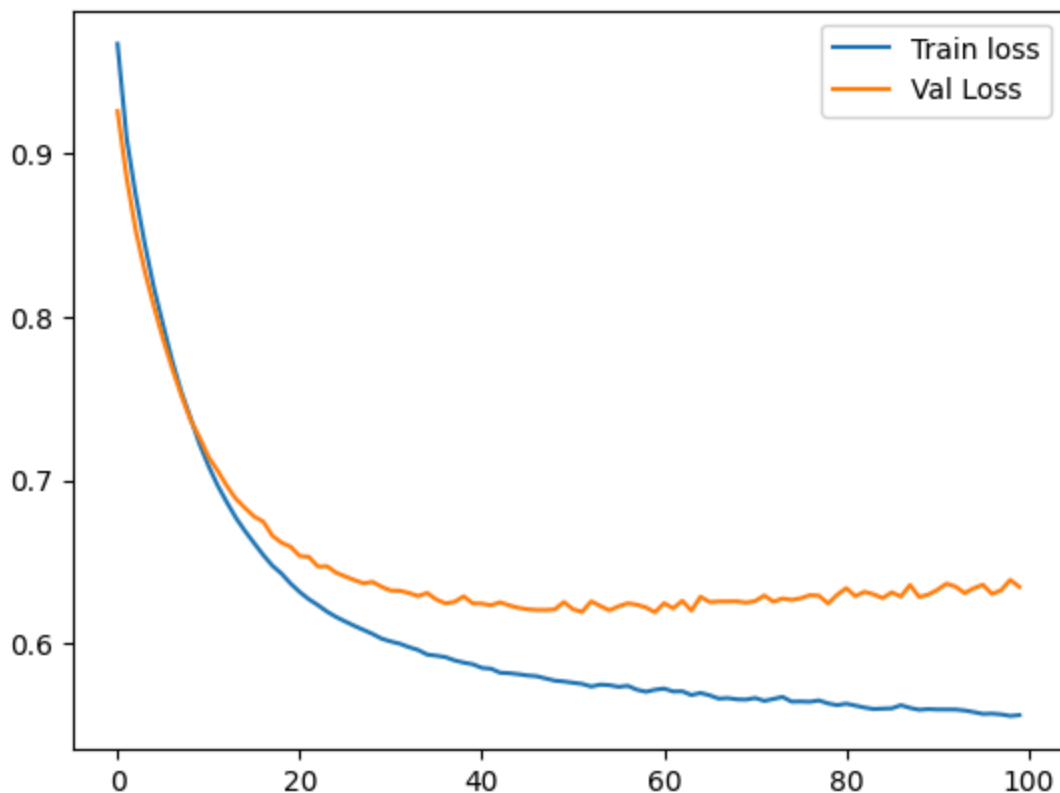
27/27 [=====] - 0s 1ms/step - loss: 0.5594 - accuracy: 0.7435 - val_loss: 0.6311 - val_accuracy: 0.6667
Epoch 95/100
27/27 [=====] - 0s 1ms/step - loss: 0.5584 - accuracy: 0.7353 - val_loss: 0.6340 - val_accuracy: 0.6714
Epoch 96/100
27/27 [=====] - 0s 1ms/step - loss: 0.5573 - accuracy: 0.7482 - val_loss: 0.6362 - val_accuracy: 0.6714
Epoch 97/100
27/27 [=====] - 0s 1ms/step - loss: 0.5575 - accuracy: 0.7471 - val_loss: 0.6305 - val_accuracy: 0.6667
Epoch 98/100
27/27 [=====] - 0s 1ms/step - loss: 0.5570 - accuracy: 0.7482 - val_loss: 0.6328 - val_accuracy: 0.6714
Epoch 99/100
27/27 [=====] - 0s 1ms/step - loss: 0.5561 - accuracy: 0.7494 - val_loss: 0.6391 - val_accuracy: 0.6714
Epoch 100/100
27/27 [=====] - 0s 1ms/step - loss: 0.5566 - accuracy: 0.7471 - val_loss: 0.6348 - val_accuracy: 0.6714

```

```

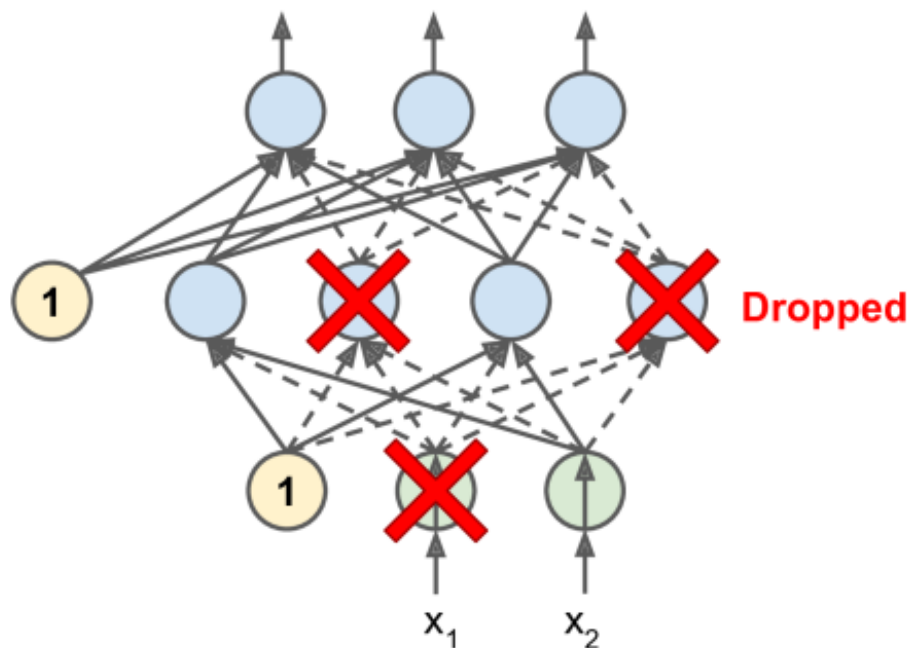
In [32]: plt.plot(history_l2.history['loss'], label='Train loss')
plt.plot(history_l2.history['val_loss'], label='Val Loss')
plt.legend();

```



Dropout

- Un algoritmo sencillo que ha mostrado ser muy efectivo al entrenar redes neuronales
- En cada iteración de entrenamiento, cada neurona (excepto las neuronas de salida) tiene una probabilidad p de ser **excluída** (ignorada).
- La probabilidad p es llamada **tasa de dropout**. Por lo general esta tasa se escoge como:
 - Entre 40%-50% para redes convolucionales
 - Entre 20%-30% para redes recurrentes
 - Para otras redes, se puede escoger valores entre el 10% hasta el 50%



```
In [33]: from tensorflow.keras.layers import Dropout
# Con Dropout
# Modelo secuencial
model = Sequential()
# Primera capa oculta
model.add(Dense(19, # Cuántas neuronas tienes en tu primera capa oculta
                  input_dim = input_shape, # ¿Cuál es la forma de tus caracter
                  activation = 'relu')) # ¿Qué función de activación estás usando
# A continuación agregamos nuestra capa de dropout
model.add(Dropout(.3))
model.add(Dense(10,
                activation = 'relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation = 'sigmoid'))
model.compile(loss = 'bce', optimizer = 'adam', metrics=['accuracy'])
history_dropout = model.fit(X_train, y_train,
                           validation_data = (X_val, y_val),
                           epochs=100)
```

Epoch 1/100
27/27 [=====] - 0s 4ms/step - loss: 0.7993 - accuracy: 0.4659 - val_loss: 0.6787 - val_accuracy: 0.5775
Epoch 2/100
27/27 [=====] - 0s 1ms/step - loss: 0.7248 - accuracy: 0.4882 - val_loss: 0.6506 - val_accuracy: 0.6479
Epoch 3/100
27/27 [=====] - 0s 1ms/step - loss: 0.6685 - accuracy: 0.5800 - val_loss: 0.6313 - val_accuracy: 0.6573
Epoch 4/100
27/27 [=====] - 0s 1ms/step - loss: 0.6539 - accuracy: 0.5929 - val_loss: 0.6175 - val_accuracy: 0.6667
Epoch 5/100
27/27 [=====] - 0s 1ms/step - loss: 0.6522 - accuracy: 0.6235 - val_loss: 0.6111 - val_accuracy: 0.6385
Epoch 6/100
27/27 [=====] - 0s 1ms/step - loss: 0.6383 - accuracy: 0.6400 - val_loss: 0.6076 - val_accuracy: 0.6291
Epoch 7/100
27/27 [=====] - 0s 2ms/step - loss: 0.6365 - accuracy: 0.6282 - val_loss: 0.6045 - val_accuracy: 0.6338
Epoch 8/100
27/27 [=====] - 0s 1ms/step - loss: 0.6180 - accuracy: 0.6506 - val_loss: 0.6028 - val_accuracy: 0.6338
Epoch 9/100
27/27 [=====] - 0s 1ms/step - loss: 0.6194 - accuracy: 0.6494 - val_loss: 0.5989 - val_accuracy: 0.6432
Epoch 10/100
27/27 [=====] - 0s 1ms/step - loss: 0.6104 - accuracy: 0.6541 - val_loss: 0.5962 - val_accuracy: 0.6432
Epoch 11/100
27/27 [=====] - 0s 1ms/step - loss: 0.6097 - accuracy: 0.6600 - val_loss: 0.5963 - val_accuracy: 0.6385
Epoch 12/100
27/27 [=====] - 0s 1ms/step - loss: 0.6032 - accuracy: 0.6612 - val_loss: 0.5938 - val_accuracy: 0.6479
Epoch 13/100
27/27 [=====] - 0s 1ms/step - loss: 0.6004 - accuracy: 0.6788 - val_loss: 0.5904 - val_accuracy: 0.6620
Epoch 14/100
27/27 [=====] - 0s 1ms/step - loss: 0.6153 - accuracy: 0.6635 - val_loss: 0.5917 - val_accuracy: 0.6432
Epoch 15/100
27/27 [=====] - 0s 1ms/step - loss: 0.6200 - accuracy: 0.6800 - val_loss: 0.5901 - val_accuracy: 0.6573
Epoch 16/100
27/27 [=====] - 0s 2ms/step - loss: 0.6006 - accuracy: 0.6882 - val_loss: 0.5899 - val_accuracy: 0.6526
Epoch 17/100
27/27 [=====] - 0s 2ms/step - loss: 0.6029 - accuracy: 0.6741 - val_loss: 0.5886 - val_accuracy: 0.6573
Epoch 18/100
27/27 [=====] - 0s 2ms/step - loss: 0.6023 - accuracy: 0.6847 - val_loss: 0.5891 - val_accuracy: 0.6667
Epoch 19/100
27/27 [=====] - 0s 2ms/step - loss: 0.5927 - accuracy:

cy: 0.6753 - val_loss: 0.5894 - val_accuracy: 0.6667
Epoch 20/100
27/27 [=====] - 0s 2ms/step - loss: 0.6089 - accuracy: 0.6541 - val_loss: 0.5895 - val_accuracy: 0.6620
Epoch 21/100
27/27 [=====] - 0s 1ms/step - loss: 0.5894 - accuracy: 0.6765 - val_loss: 0.5916 - val_accuracy: 0.6479
Epoch 22/100
27/27 [=====] - 0s 1ms/step - loss: 0.5973 - accuracy: 0.6788 - val_loss: 0.5901 - val_accuracy: 0.6526
Epoch 23/100
27/27 [=====] - 0s 1ms/step - loss: 0.5964 - accuracy: 0.6882 - val_loss: 0.5872 - val_accuracy: 0.6620
Epoch 24/100
27/27 [=====] - 0s 1ms/step - loss: 0.6228 - accuracy: 0.6859 - val_loss: 0.5897 - val_accuracy: 0.6573
Epoch 25/100
27/27 [=====] - 0s 1ms/step - loss: 0.5846 - accuracy: 0.7059 - val_loss: 0.5933 - val_accuracy: 0.6385
Epoch 26/100
27/27 [=====] - 0s 1ms/step - loss: 0.6001 - accuracy: 0.6718 - val_loss: 0.5922 - val_accuracy: 0.6385
Epoch 27/100
27/27 [=====] - 0s 1ms/step - loss: 0.5980 - accuracy: 0.6694 - val_loss: 0.5911 - val_accuracy: 0.6385
Epoch 28/100
27/27 [=====] - 0s 1ms/step - loss: 0.6164 - accuracy: 0.6824 - val_loss: 0.5917 - val_accuracy: 0.6385
Epoch 29/100
27/27 [=====] - 0s 1ms/step - loss: 0.6043 - accuracy: 0.6859 - val_loss: 0.5910 - val_accuracy: 0.6385
Epoch 30/100
27/27 [=====] - 0s 1ms/step - loss: 0.5809 - accuracy: 0.7012 - val_loss: 0.5887 - val_accuracy: 0.6479
Epoch 31/100
27/27 [=====] - 0s 2ms/step - loss: 0.5904 - accuracy: 0.6988 - val_loss: 0.5874 - val_accuracy: 0.6479
Epoch 32/100
27/27 [=====] - 0s 2ms/step - loss: 0.5869 - accuracy: 0.6765 - val_loss: 0.5882 - val_accuracy: 0.6385
Epoch 33/100
27/27 [=====] - 0s 2ms/step - loss: 0.5850 - accuracy: 0.6929 - val_loss: 0.5875 - val_accuracy: 0.6479
Epoch 34/100
27/27 [=====] - 0s 1ms/step - loss: 0.5873 - accuracy: 0.6918 - val_loss: 0.5886 - val_accuracy: 0.6573
Epoch 35/100
27/27 [=====] - 0s 1ms/step - loss: 0.5875 - accuracy: 0.6894 - val_loss: 0.5874 - val_accuracy: 0.6479
Epoch 36/100
27/27 [=====] - 0s 1ms/step - loss: 0.5846 - accuracy: 0.7024 - val_loss: 0.5861 - val_accuracy: 0.6526
Epoch 37/100
27/27 [=====] - 0s 1ms/step - loss: 0.5911 - accuracy: 0.7094 - val_loss: 0.5874 - val_accuracy: 0.6573
Epoch 38/100

27/27 [=====] - 0s 1ms/step - loss: 0.5832 - accuracy: 0.6906 - val_loss: 0.5862 - val_accuracy: 0.6667
Epoch 39/100

27/27 [=====] - 0s 1ms/step - loss: 0.5946 - accuracy: 0.6965 - val_loss: 0.5869 - val_accuracy: 0.6714
Epoch 40/100

27/27 [=====] - 0s 1ms/step - loss: 0.5900 - accuracy: 0.6988 - val_loss: 0.5880 - val_accuracy: 0.6620
Epoch 41/100

27/27 [=====] - 0s 1ms/step - loss: 0.5977 - accuracy: 0.6906 - val_loss: 0.5873 - val_accuracy: 0.6573
Epoch 42/100

27/27 [=====] - 0s 1ms/step - loss: 0.5892 - accuracy: 0.6859 - val_loss: 0.5859 - val_accuracy: 0.6667
Epoch 43/100

27/27 [=====] - 0s 1ms/step - loss: 0.5697 - accuracy: 0.6941 - val_loss: 0.5870 - val_accuracy: 0.6620
Epoch 44/100

27/27 [=====] - 0s 1ms/step - loss: 0.5863 - accuracy: 0.6976 - val_loss: 0.5875 - val_accuracy: 0.6667
Epoch 45/100

27/27 [=====] - 0s 1ms/step - loss: 0.5770 - accuracy: 0.6988 - val_loss: 0.5886 - val_accuracy: 0.6620
Epoch 46/100

27/27 [=====] - 0s 1ms/step - loss: 0.5755 - accuracy: 0.7035 - val_loss: 0.5893 - val_accuracy: 0.6667
Epoch 47/100

27/27 [=====] - 0s 1ms/step - loss: 0.5861 - accuracy: 0.6753 - val_loss: 0.5877 - val_accuracy: 0.6667
Epoch 48/100

27/27 [=====] - 0s 1ms/step - loss: 0.6017 - accuracy: 0.6765 - val_loss: 0.5909 - val_accuracy: 0.6526
Epoch 49/100

27/27 [=====] - 0s 1ms/step - loss: 0.5857 - accuracy: 0.7059 - val_loss: 0.5894 - val_accuracy: 0.6667
Epoch 50/100

27/27 [=====] - 0s 1ms/step - loss: 0.5765 - accuracy: 0.6941 - val_loss: 0.5899 - val_accuracy: 0.6620
Epoch 51/100

27/27 [=====] - 0s 1ms/step - loss: 0.5783 - accuracy: 0.6929 - val_loss: 0.5916 - val_accuracy: 0.6667
Epoch 52/100

27/27 [=====] - 0s 1ms/step - loss: 0.5655 - accuracy: 0.7106 - val_loss: 0.5887 - val_accuracy: 0.6620
Epoch 53/100

27/27 [=====] - 0s 1ms/step - loss: 0.5789 - accuracy: 0.6976 - val_loss: 0.5895 - val_accuracy: 0.6573
Epoch 54/100

27/27 [=====] - 0s 1ms/step - loss: 0.5863 - accuracy: 0.6847 - val_loss: 0.5920 - val_accuracy: 0.6620
Epoch 55/100

27/27 [=====] - 0s 1ms/step - loss: 0.5947 - accuracy: 0.6918 - val_loss: 0.5925 - val_accuracy: 0.6620
Epoch 56/100

27/27 [=====] - 0s 2ms/step - loss: 0.5780 - accuracy: 0.6929 - val_loss: 0.5904 - val_accuracy: 0.6620

Epoch 57/100
27/27 [=====] - 0s 2ms/step - loss: 0.5743 - accuracy: 0.7106 - val_loss: 0.5910 - val_accuracy: 0.6667
Epoch 58/100
27/27 [=====] - 0s 2ms/step - loss: 0.5848 - accuracy: 0.6929 - val_loss: 0.5904 - val_accuracy: 0.6573
Epoch 59/100
27/27 [=====] - 0s 1ms/step - loss: 0.5788 - accuracy: 0.7047 - val_loss: 0.5914 - val_accuracy: 0.6667
Epoch 60/100
27/27 [=====] - 0s 1ms/step - loss: 0.5831 - accuracy: 0.7153 - val_loss: 0.5915 - val_accuracy: 0.6573
Epoch 61/100
27/27 [=====] - 0s 1ms/step - loss: 0.5794 - accuracy: 0.6976 - val_loss: 0.5883 - val_accuracy: 0.6573
Epoch 62/100
27/27 [=====] - 0s 2ms/step - loss: 0.5650 - accuracy: 0.6988 - val_loss: 0.5894 - val_accuracy: 0.6667
Epoch 63/100
27/27 [=====] - 0s 1ms/step - loss: 0.5726 - accuracy: 0.6988 - val_loss: 0.5887 - val_accuracy: 0.6620
Epoch 64/100
27/27 [=====] - 0s 1ms/step - loss: 0.5743 - accuracy: 0.7082 - val_loss: 0.5895 - val_accuracy: 0.6667
Epoch 65/100
27/27 [=====] - 0s 1ms/step - loss: 0.5732 - accuracy: 0.7082 - val_loss: 0.5890 - val_accuracy: 0.6714
Epoch 66/100
27/27 [=====] - 0s 1ms/step - loss: 0.5851 - accuracy: 0.6965 - val_loss: 0.5878 - val_accuracy: 0.6620
Epoch 67/100
27/27 [=====] - 0s 1ms/step - loss: 0.5774 - accuracy: 0.7012 - val_loss: 0.5903 - val_accuracy: 0.6620
Epoch 68/100
27/27 [=====] - 0s 1ms/step - loss: 0.5551 - accuracy: 0.7141 - val_loss: 0.5876 - val_accuracy: 0.6808
Epoch 69/100
27/27 [=====] - 0s 1ms/step - loss: 0.5646 - accuracy: 0.7071 - val_loss: 0.5857 - val_accuracy: 0.6620
Epoch 70/100
27/27 [=====] - 0s 2ms/step - loss: 0.5763 - accuracy: 0.6988 - val_loss: 0.5883 - val_accuracy: 0.6620
Epoch 71/100
27/27 [=====] - 0s 2ms/step - loss: 0.5714 - accuracy: 0.6941 - val_loss: 0.5895 - val_accuracy: 0.6667
Epoch 72/100
27/27 [=====] - 0s 2ms/step - loss: 0.5689 - accuracy: 0.7024 - val_loss: 0.5894 - val_accuracy: 0.6714
Epoch 73/100
27/27 [=====] - 0s 1ms/step - loss: 0.5750 - accuracy: 0.6976 - val_loss: 0.5913 - val_accuracy: 0.6761
Epoch 74/100
27/27 [=====] - 0s 1ms/step - loss: 0.5733 - accuracy: 0.6953 - val_loss: 0.5912 - val_accuracy: 0.6714
Epoch 75/100
27/27 [=====] - 0s 1ms/step - loss: 0.5757 - accuracy:

cy: 0.6918 - val_loss: 0.5893 - val_accuracy: 0.6667
Epoch 76/100
27/27 [=====] - 0s 1ms/step - loss: 0.5760 - accuracy: 0.7176 - val_loss: 0.5875 - val_accuracy: 0.6761
Epoch 77/100
27/27 [=====] - 0s 1ms/step - loss: 0.5819 - accuracy: 0.6882 - val_loss: 0.5879 - val_accuracy: 0.6714
Epoch 78/100
27/27 [=====] - 0s 1ms/step - loss: 0.5741 - accuracy: 0.7200 - val_loss: 0.5885 - val_accuracy: 0.6761
Epoch 79/100
27/27 [=====] - 0s 1ms/step - loss: 0.5706 - accuracy: 0.6918 - val_loss: 0.5883 - val_accuracy: 0.6761
Epoch 80/100
27/27 [=====] - 0s 1ms/step - loss: 0.5563 - accuracy: 0.7094 - val_loss: 0.5868 - val_accuracy: 0.6761
Epoch 81/100
27/27 [=====] - 0s 1ms/step - loss: 0.5711 - accuracy: 0.7012 - val_loss: 0.5905 - val_accuracy: 0.6714
Epoch 82/100
27/27 [=====] - 0s 2ms/step - loss: 0.5677 - accuracy: 0.7094 - val_loss: 0.5902 - val_accuracy: 0.6714
Epoch 83/100
27/27 [=====] - 0s 1ms/step - loss: 0.5585 - accuracy: 0.6988 - val_loss: 0.5890 - val_accuracy: 0.6714
Epoch 84/100
27/27 [=====] - 0s 1ms/step - loss: 0.5787 - accuracy: 0.7106 - val_loss: 0.5916 - val_accuracy: 0.6714
Epoch 85/100
27/27 [=====] - 0s 1ms/step - loss: 0.5664 - accuracy: 0.7141 - val_loss: 0.5906 - val_accuracy: 0.6808
Epoch 86/100
27/27 [=====] - 0s 1ms/step - loss: 0.5609 - accuracy: 0.7082 - val_loss: 0.5910 - val_accuracy: 0.6761
Epoch 87/100
27/27 [=====] - 0s 2ms/step - loss: 0.5810 - accuracy: 0.6894 - val_loss: 0.5886 - val_accuracy: 0.6854
Epoch 88/100
27/27 [=====] - 0s 2ms/step - loss: 0.5655 - accuracy: 0.7094 - val_loss: 0.5874 - val_accuracy: 0.6854
Epoch 89/100
27/27 [=====] - 0s 2ms/step - loss: 0.5654 - accuracy: 0.7118 - val_loss: 0.5888 - val_accuracy: 0.6761
Epoch 90/100
27/27 [=====] - 0s 2ms/step - loss: 0.5620 - accuracy: 0.7129 - val_loss: 0.5875 - val_accuracy: 0.6714
Epoch 91/100
27/27 [=====] - 0s 1ms/step - loss: 0.5701 - accuracy: 0.7141 - val_loss: 0.5894 - val_accuracy: 0.6854
Epoch 92/100
27/27 [=====] - 0s 1ms/step - loss: 0.5683 - accuracy: 0.7082 - val_loss: 0.5897 - val_accuracy: 0.6761
Epoch 93/100
27/27 [=====] - 0s 1ms/step - loss: 0.5623 - accuracy: 0.7235 - val_loss: 0.5880 - val_accuracy: 0.6808
Epoch 94/100

```

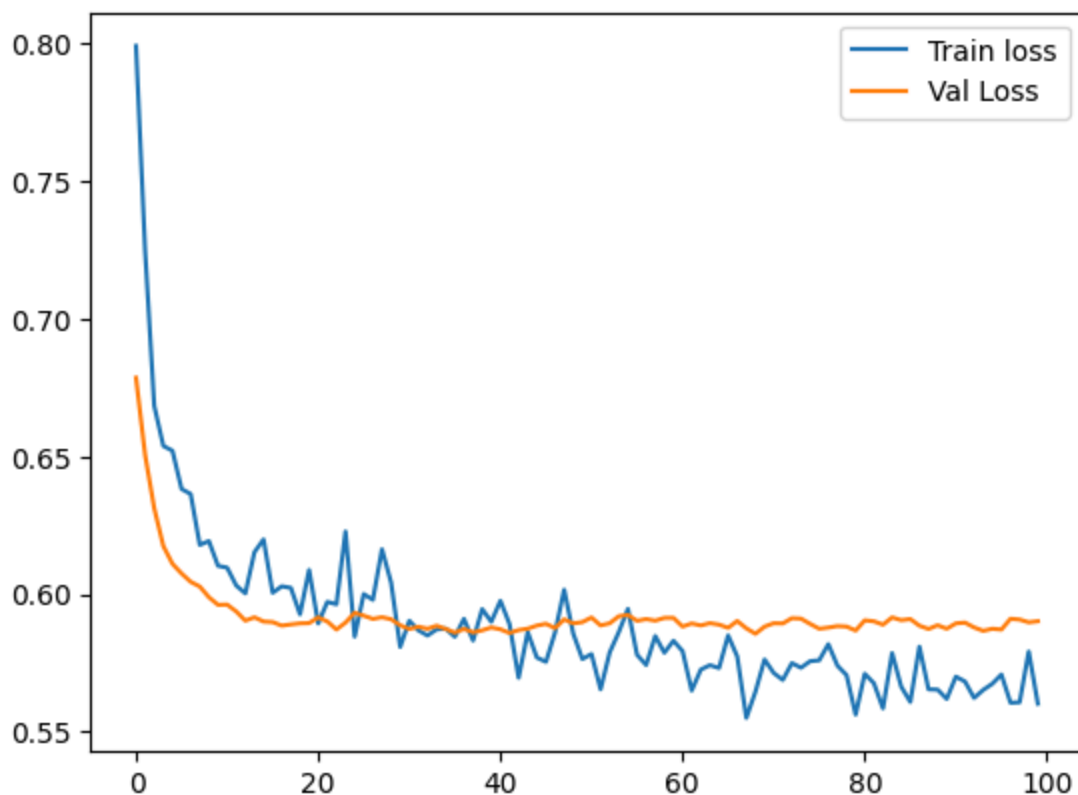
27/27 [=====] - 0s 1ms/step - loss: 0.5652 - accuracy: 0.7094 - val_loss: 0.5867 - val_accuracy: 0.6808
Epoch 95/100
27/27 [=====] - 0s 1ms/step - loss: 0.5675 - accuracy: 0.7094 - val_loss: 0.5875 - val_accuracy: 0.6854
Epoch 96/100
27/27 [=====] - 0s 1ms/step - loss: 0.5709 - accuracy: 0.7188 - val_loss: 0.5872 - val_accuracy: 0.6901
Epoch 97/100
27/27 [=====] - 0s 2ms/step - loss: 0.5606 - accuracy: 0.7235 - val_loss: 0.5911 - val_accuracy: 0.6761
Epoch 98/100
27/27 [=====] - 0s 1ms/step - loss: 0.5608 - accuracy: 0.7035 - val_loss: 0.5908 - val_accuracy: 0.6901
Epoch 99/100
27/27 [=====] - 0s 1ms/step - loss: 0.5792 - accuracy: 0.7118 - val_loss: 0.5898 - val_accuracy: 0.6620
Epoch 100/100
27/27 [=====] - 0s 2ms/step - loss: 0.5603 - accuracy: 0.7106 - val_loss: 0.5902 - val_accuracy: 0.6667

```

```

In [34]: plt.plot(history_dropout.history['loss'], label='Train loss')
plt.plot(history_dropout.history['val_loss'], label='Val Loss')
plt.legend();

```



Early stop (detención temprana)

Epoch 1/100
27/27 [=====] - 0s 5ms/step - loss: 1.0442 - accuracy: 0.3847 - val_loss: 0.9115 - val_accuracy: 0.3383
Epoch 2/100
27/27 [=====] - 0s 1ms/step - loss: 0.8093 - accuracy: 0.3906 - val_loss: 0.7688 - val_accuracy: 0.3684
Epoch 3/100
27/27 [=====] - 0s 1ms/step - loss: 0.7163 - accuracy: 0.4647 - val_loss: 0.6940 - val_accuracy: 0.4925
Epoch 4/100
27/27 [=====] - 0s 1ms/step - loss: 0.6666 - accuracy: 0.5776 - val_loss: 0.6497 - val_accuracy: 0.5977
Epoch 5/100
27/27 [=====] - 0s 1ms/step - loss: 0.6369 - accuracy: 0.6365 - val_loss: 0.6177 - val_accuracy: 0.6654
Epoch 6/100
27/27 [=====] - 0s 1ms/step - loss: 0.6173 - accuracy: 0.6765 - val_loss: 0.5987 - val_accuracy: 0.6992
Epoch 7/100
27/27 [=====] - 0s 1ms/step - loss: 0.6044 - accuracy: 0.6847 - val_loss: 0.5839 - val_accuracy: 0.7218
Epoch 8/100
27/27 [=====] - 0s 2ms/step - loss: 0.5953 - accuracy: 0.6859 - val_loss: 0.5713 - val_accuracy: 0.7594
Epoch 9/100
27/27 [=====] - 0s 2ms/step - loss: 0.5874 - accuracy: 0.6988 - val_loss: 0.5623 - val_accuracy: 0.7444
Epoch 10/100
27/27 [=====] - 0s 2ms/step - loss: 0.5812 - accuracy: 0.7082 - val_loss: 0.5552 - val_accuracy: 0.7406
Epoch 11/100
27/27 [=====] - 0s 2ms/step - loss: 0.5773 - accuracy: 0.7024 - val_loss: 0.5501 - val_accuracy: 0.7556
Epoch 12/100
27/27 [=====] - 0s 1ms/step - loss: 0.5729 - accuracy: 0.7012 - val_loss: 0.5467 - val_accuracy: 0.7406
Epoch 13/100
27/27 [=====] - 0s 1ms/step - loss: 0.5701 - accuracy: 0.7024 - val_loss: 0.5435 - val_accuracy: 0.7556
Epoch 14/100
27/27 [=====] - 0s 1ms/step - loss: 0.5671 - accuracy: 0.7082 - val_loss: 0.5400 - val_accuracy: 0.7444
Epoch 15/100
27/27 [=====] - 0s 1ms/step - loss: 0.5652 - accuracy: 0.7047 - val_loss: 0.5393 - val_accuracy: 0.7444
Epoch 16/100
27/27 [=====] - 0s 1ms/step - loss: 0.5634 - accuracy: 0.7082 - val_loss: 0.5371 - val_accuracy: 0.7556
Epoch 17/100
27/27 [=====] - 0s 1ms/step - loss: 0.5616 - accuracy: 0.7118 - val_loss: 0.5357 - val_accuracy: 0.7481
Epoch 18/100
27/27 [=====] - 0s 1ms/step - loss: 0.5601 - accuracy: 0.7129 - val_loss: 0.5344 - val_accuracy: 0.7481
Epoch 19/100
27/27 [=====] - 0s 1ms/step - loss: 0.5589 - accuracy:

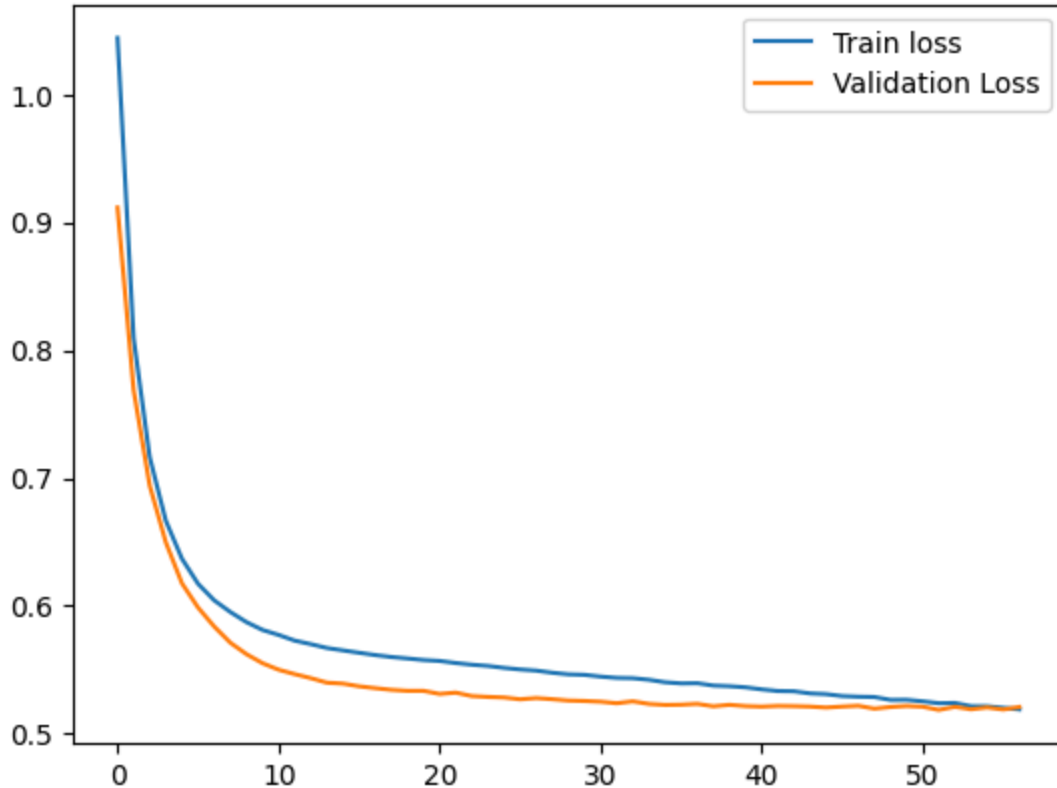
cy: 0.7059 - val_loss: 0.5336 - val_accuracy: 0.7481
Epoch 20/100
27/27 [=====] - 0s 1ms/step - loss: 0.5577 - accuracy: 0.7212 - val_loss: 0.5336 - val_accuracy: 0.7444
Epoch 21/100
27/27 [=====] - 0s 2ms/step - loss: 0.5570 - accuracy: 0.7153 - val_loss: 0.5312 - val_accuracy: 0.7556
Epoch 22/100
27/27 [=====] - 0s 2ms/step - loss: 0.5554 - accuracy: 0.7212 - val_loss: 0.5322 - val_accuracy: 0.7444
Epoch 23/100
27/27 [=====] - 0s 2ms/step - loss: 0.5540 - accuracy: 0.7153 - val_loss: 0.5295 - val_accuracy: 0.7481
Epoch 24/100
27/27 [=====] - 0s 1ms/step - loss: 0.5531 - accuracy: 0.7235 - val_loss: 0.5288 - val_accuracy: 0.7481
Epoch 25/100
27/27 [=====] - 0s 1ms/step - loss: 0.5516 - accuracy: 0.7259 - val_loss: 0.5284 - val_accuracy: 0.7406
Epoch 26/100
27/27 [=====] - 0s 1ms/step - loss: 0.5503 - accuracy: 0.7306 - val_loss: 0.5270 - val_accuracy: 0.7406
Epoch 27/100
27/27 [=====] - 0s 1ms/step - loss: 0.5494 - accuracy: 0.7294 - val_loss: 0.5279 - val_accuracy: 0.7368
Epoch 28/100
27/27 [=====] - 0s 1ms/step - loss: 0.5478 - accuracy: 0.7318 - val_loss: 0.5271 - val_accuracy: 0.7368
Epoch 29/100
27/27 [=====] - 0s 1ms/step - loss: 0.5465 - accuracy: 0.7294 - val_loss: 0.5260 - val_accuracy: 0.7444
Epoch 30/100
27/27 [=====] - 0s 1ms/step - loss: 0.5461 - accuracy: 0.7329 - val_loss: 0.5257 - val_accuracy: 0.7368
Epoch 31/100
27/27 [=====] - 0s 1ms/step - loss: 0.5447 - accuracy: 0.7306 - val_loss: 0.5251 - val_accuracy: 0.7406
Epoch 32/100
27/27 [=====] - 0s 1ms/step - loss: 0.5437 - accuracy: 0.7282 - val_loss: 0.5239 - val_accuracy: 0.7368
Epoch 33/100
27/27 [=====] - 0s 1ms/step - loss: 0.5435 - accuracy: 0.7306 - val_loss: 0.5254 - val_accuracy: 0.7368
Epoch 34/100
27/27 [=====] - 0s 1ms/step - loss: 0.5423 - accuracy: 0.7247 - val_loss: 0.5234 - val_accuracy: 0.7368
Epoch 35/100
27/27 [=====] - 0s 1ms/step - loss: 0.5403 - accuracy: 0.7259 - val_loss: 0.5226 - val_accuracy: 0.7406
Epoch 36/100
27/27 [=====] - 0s 1ms/step - loss: 0.5394 - accuracy: 0.7294 - val_loss: 0.5228 - val_accuracy: 0.7331
Epoch 37/100
27/27 [=====] - 0s 1ms/step - loss: 0.5396 - accuracy: 0.7306 - val_loss: 0.5234 - val_accuracy: 0.7368
Epoch 38/100

27/27 [=====] - 0s 1ms/step - loss: 0.5377 - accuracy: 0.7224 - val_loss: 0.5215 - val_accuracy: 0.7368
Epoch 39/100
27/27 [=====] - 0s 1ms/step - loss: 0.5372 - accuracy: 0.7247 - val_loss: 0.5227 - val_accuracy: 0.7331
Epoch 40/100
27/27 [=====] - 0s 1ms/step - loss: 0.5364 - accuracy: 0.7353 - val_loss: 0.5217 - val_accuracy: 0.7331
Epoch 41/100
27/27 [=====] - 0s 1ms/step - loss: 0.5347 - accuracy: 0.7318 - val_loss: 0.5213 - val_accuracy: 0.7368
Epoch 42/100
27/27 [=====] - 0s 1ms/step - loss: 0.5335 - accuracy: 0.7365 - val_loss: 0.5218 - val_accuracy: 0.7293
Epoch 43/100
27/27 [=====] - 0s 1ms/step - loss: 0.5333 - accuracy: 0.7318 - val_loss: 0.5216 - val_accuracy: 0.7331
Epoch 44/100
27/27 [=====] - 0s 1ms/step - loss: 0.5316 - accuracy: 0.7388 - val_loss: 0.5213 - val_accuracy: 0.7293
Epoch 45/100
27/27 [=====] - 0s 1ms/step - loss: 0.5310 - accuracy: 0.7365 - val_loss: 0.5206 - val_accuracy: 0.7293
Epoch 46/100
27/27 [=====] - 0s 1ms/step - loss: 0.5295 - accuracy: 0.7365 - val_loss: 0.5213 - val_accuracy: 0.7293
Epoch 47/100
27/27 [=====] - 0s 1ms/step - loss: 0.5290 - accuracy: 0.7435 - val_loss: 0.5219 - val_accuracy: 0.7406
Epoch 48/100
27/27 [=====] - 0s 1ms/step - loss: 0.5288 - accuracy: 0.7353 - val_loss: 0.5197 - val_accuracy: 0.7331
Epoch 49/100
27/27 [=====] - 0s 1ms/step - loss: 0.5265 - accuracy: 0.7459 - val_loss: 0.5210 - val_accuracy: 0.7331
Epoch 50/100
27/27 [=====] - 0s 1ms/step - loss: 0.5266 - accuracy: 0.7435 - val_loss: 0.5217 - val_accuracy: 0.7406
Epoch 51/100
27/27 [=====] - 0s 1ms/step - loss: 0.5254 - accuracy: 0.7471 - val_loss: 0.5212 - val_accuracy: 0.7519
Epoch 52/100
27/27 [=====] - 0s 1ms/step - loss: 0.5239 - accuracy: 0.7459 - val_loss: 0.5187 - val_accuracy: 0.7444
Epoch 53/100
27/27 [=====] - 0s 1ms/step - loss: 0.5241 - accuracy: 0.7459 - val_loss: 0.5211 - val_accuracy: 0.7406
Epoch 54/100
27/27 [=====] - 0s 1ms/step - loss: 0.5216 - accuracy: 0.7494 - val_loss: 0.5192 - val_accuracy: 0.7406
Epoch 55/100
27/27 [=====] - 0s 1ms/step - loss: 0.5215 - accuracy: 0.7447 - val_loss: 0.5206 - val_accuracy: 0.7444
Epoch 56/100
27/27 [=====] - 0s 2ms/step - loss: 0.5203 - accuracy: 0.7506 - val_loss: 0.5190 - val_accuracy: 0.7481

Epoch 57/100

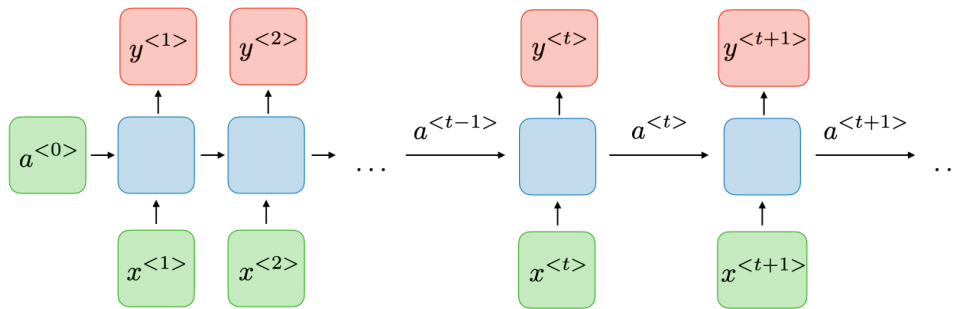
27/27 [=====] - 0s 2ms/step - loss: 0.5191 - accuracy: 0.7506 - val_loss: 0.5209 - val_accuracy: 0.7519

```
In [37]: plt.plot(history_earlystop.history['loss'], label='Train loss')
plt.plot(history_earlystop.history['val_loss'], label='Validation Loss')
plt.legend();
```



Redes Neuronales Recurrentes (RNN- Recurrent Neural Networks)

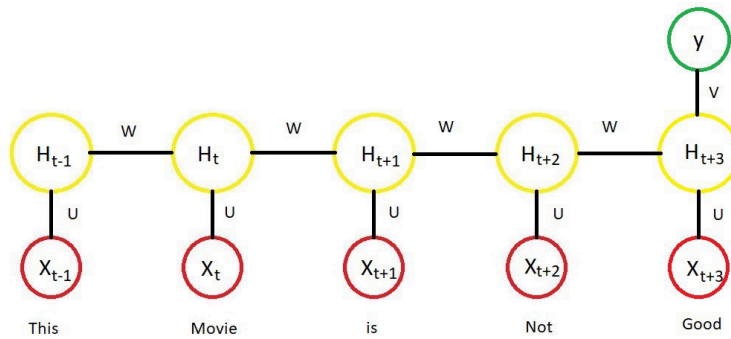
- Hasta el momento hemos trabajado con redes FeedForward, donde las activaciones fluyen en una dirección: desde la capa de entrada a la capa de salida.
- Las redes neuronales recurrentes son similares a las redes FeedForward, salvo que también existen conexiones en el sentido inverso.
- Las redes neuronales recurrentes son utilizadas generalmente para procesar secuencias de datos temporalmente relacionados, pues toman en cuenta lo que ocurrió previamente
- Las unidades de procesamiento que normalmente se utilizan son
 - Long Short Term Memory (LSTM)
 - Gated Recurrent Unit (GRU)
- Las redes Neuronales Recurrentes son muy utilizadas para predicciones de señales y en aplicaciones de NLP (Natural Language Processing)



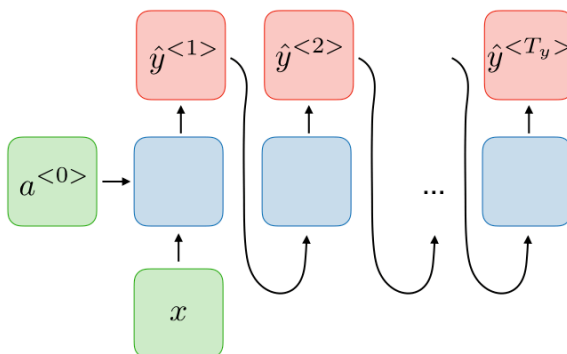
Fuente: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>

Tipos de arquitecturas de Redes Neuronales Recurrentes

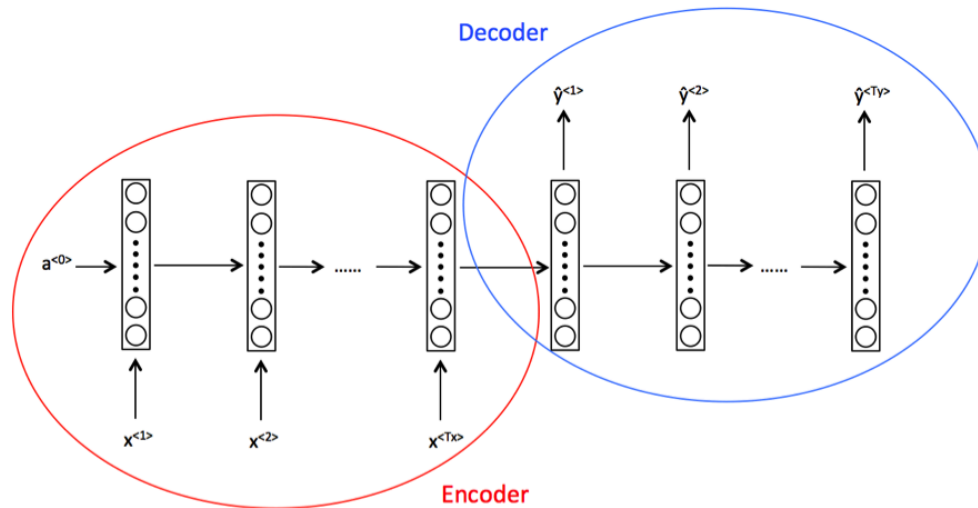
- **Many to One:** Múltiples entradas son usadas para la obtención de una salida



- **One to Many:** Una única entrada produce múltiples salidas



- **Many to many:** Múltiples entradas producen múltiples salidas



Fuente: <https://medium.com/analytics-vidhya/natural-language-processing-from-basics-to-using-rnn-and-lstm-ef6779e4ae66>

Ejemplo de LSTM

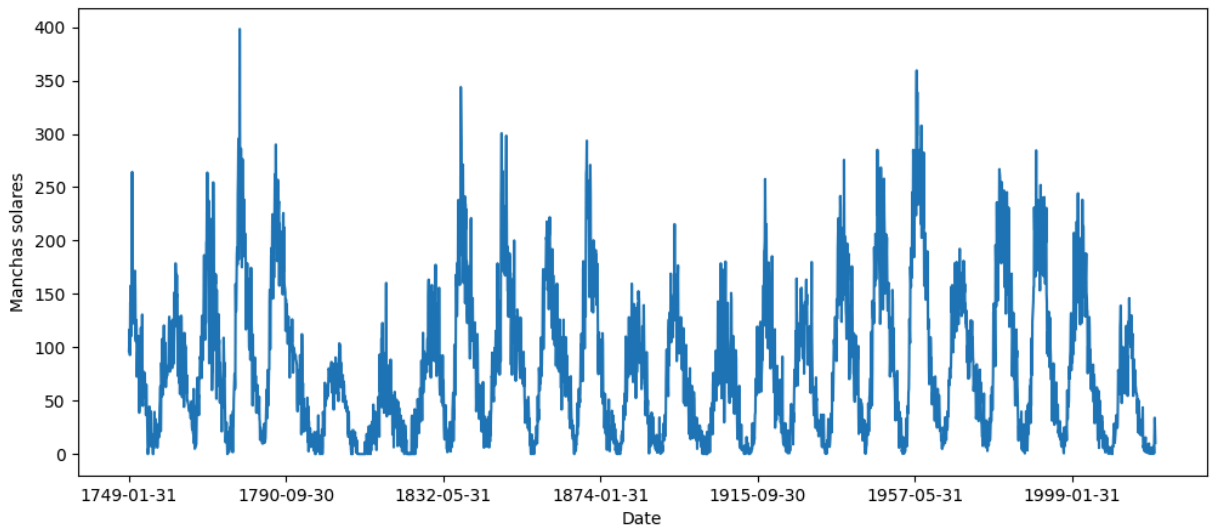
```
In [38]: # Dataset de manchas solares disponible en https://github.com/jbrownlee/Data
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM

df = pd.read_csv('data/sunspots.csv',
                 index_col = 'Date')
df = df.drop(columns=['id'])
df.rename(columns={'Monthly Mean Total Sunspot Number': 'sunspots'}, inplace=True)
df.head()
```

Out[38]:

sunspots	
Date	
1749-01-31	96.7
1749-02-28	104.3
1749-03-31	116.7
1749-04-30	92.8
1749-05-31	141.7

```
In [39]: plt.figure(figsize = (12, 5))
df['sunspots'].plot()
plt.ylabel('Manchas solares');
```



```
In [40]: # Division train/test
train_values = np.array(df.loc[:'2016-12-31', 'sunspots'])
test_values = np.array(df.loc['2017-01-31':, 'sunspots'])

# Escalamiento
scaler = MinMaxScaler(feature_range = (0, 1))
train = scaler.fit_transform(train_values.reshape(-1, 1))
test = scaler.transform(test_values.reshape(-1, 1))

# Conversión a datos con tres columnas
from tensorflow.keras.preprocessing import timeseries_dataset_from_array
prev_months = 1
generator = timeseries_dataset_from_array(
    train,
    train,
    sequence_length=prev_months)
```

```
In [41]: # Modelo secuencial
model = Sequential()
# Capa LSTM
model.add(LSTM(units = 50,
                return_sequences = False # True si la siguiente capa será una
                ))
# Capa de salida
model.add(Dense(units = 1, activation = 'relu'))

# Compilado
model.compile(loss = 'mean_squared_error', optimizer = 'adam')

history_lstm = model.fit(generator, epochs=50) # el generador creará los dat

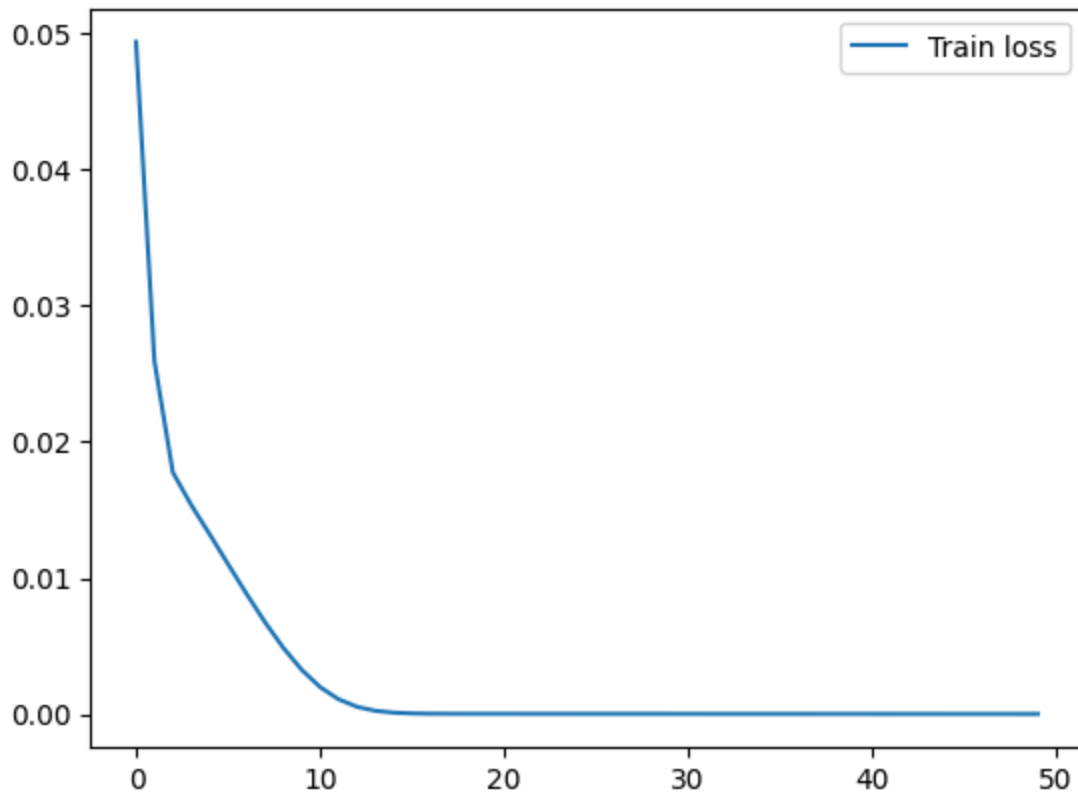
plt.plot(history_lstm.history['loss'], label='Train loss')
plt.legend()
```

```
Epoch 1/50
26/26 [=====] - 1s 2ms/step - loss: 0.0494
Epoch 2/50
26/26 [=====] - 0s 2ms/step - loss: 0.0259
Epoch 3/50
26/26 [=====] - 0s 2ms/step - loss: 0.0178
Epoch 4/50
26/26 [=====] - 0s 2ms/step - loss: 0.0154
Epoch 5/50
26/26 [=====] - 0s 2ms/step - loss: 0.0132
Epoch 6/50
26/26 [=====] - 0s 2ms/step - loss: 0.0110
Epoch 7/50
26/26 [=====] - 0s 2ms/step - loss: 0.0089
Epoch 8/50
26/26 [=====] - 0s 2ms/step - loss: 0.0068
Epoch 9/50
26/26 [=====] - 0s 2ms/step - loss: 0.0049
Epoch 10/50
26/26 [=====] - 0s 2ms/step - loss: 0.0033
Epoch 11/50
26/26 [=====] - 0s 2ms/step - loss: 0.0020
Epoch 12/50
26/26 [=====] - 0s 2ms/step - loss: 0.0011
Epoch 13/50
26/26 [=====] - 0s 2ms/step - loss: 5.4424e-04
Epoch 14/50
26/26 [=====] - 0s 2ms/step - loss: 2.4655e-04
Epoch 15/50
26/26 [=====] - 0s 3ms/step - loss: 1.0959e-04
Epoch 16/50
26/26 [=====] - 0s 2ms/step - loss: 5.5605e-05
Epoch 17/50
26/26 [=====] - 0s 2ms/step - loss: 3.7137e-05
Epoch 18/50
26/26 [=====] - 0s 2ms/step - loss: 3.1454e-05
Epoch 19/50
26/26 [=====] - 0s 2ms/step - loss: 2.9714e-05
Epoch 20/50
26/26 [=====] - 0s 2ms/step - loss: 2.9033e-05
Epoch 21/50
26/26 [=====] - 0s 2ms/step - loss: 2.8600e-05
Epoch 22/50
26/26 [=====] - 0s 2ms/step - loss: 2.8216e-05
Epoch 23/50
26/26 [=====] - 0s 2ms/step - loss: 2.7837e-05
Epoch 24/50
26/26 [=====] - 0s 3ms/step - loss: 2.7453e-05
Epoch 25/50
26/26 [=====] - 0s 2ms/step - loss: 2.7065e-05
Epoch 26/50
26/26 [=====] - 0s 2ms/step - loss: 2.6671e-05
Epoch 27/50
26/26 [=====] - 0s 2ms/step - loss: 2.6272e-05
Epoch 28/50
26/26 [=====] - 0s 2ms/step - loss: 2.5869e-05
```



```
Epoch 29/50
26/26 [=====] - 0s 2ms/step - loss: 2.5461e-05
Epoch 30/50
26/26 [=====] - 0s 2ms/step - loss: 2.5050e-05
Epoch 31/50
26/26 [=====] - 0s 2ms/step - loss: 2.4635e-05
Epoch 32/50
26/26 [=====] - 0s 2ms/step - loss: 2.4217e-05
Epoch 33/50
26/26 [=====] - 0s 2ms/step - loss: 2.3796e-05
Epoch 34/50
26/26 [=====] - 0s 2ms/step - loss: 2.3372e-05
Epoch 35/50
26/26 [=====] - 0s 2ms/step - loss: 2.2947e-05
Epoch 36/50
26/26 [=====] - 0s 2ms/step - loss: 2.2519e-05
Epoch 37/50
26/26 [=====] - 0s 2ms/step - loss: 2.2090e-05
Epoch 38/50
26/26 [=====] - 0s 3ms/step - loss: 2.1660e-05
Epoch 39/50
26/26 [=====] - 0s 2ms/step - loss: 2.1229e-05
Epoch 40/50
26/26 [=====] - 0s 2ms/step - loss: 2.0797e-05
Epoch 41/50
26/26 [=====] - 0s 2ms/step - loss: 2.0365e-05
Epoch 42/50
26/26 [=====] - 0s 2ms/step - loss: 1.9933e-05
Epoch 43/50
26/26 [=====] - 0s 2ms/step - loss: 1.9500e-05
Epoch 44/50
26/26 [=====] - 0s 2ms/step - loss: 1.9069e-05
Epoch 45/50
26/26 [=====] - 0s 2ms/step - loss: 1.8638e-05
Epoch 46/50
26/26 [=====] - 0s 2ms/step - loss: 1.8208e-05
Epoch 47/50
26/26 [=====] - 0s 2ms/step - loss: 1.7779e-05
Epoch 48/50
26/26 [=====] - 0s 2ms/step - loss: 1.7352e-05
Epoch 49/50
26/26 [=====] - 0s 2ms/step - loss: 1.6926e-05
Epoch 50/50
26/26 [=====] - 0s 2ms/step - loss: 1.6503e-05
```

```
Out[41]: <matplotlib.legend.Legend at 0x7fae470de170>
```



In [42]: `model.summary()`

Model: "sequential_5"

Layer (type)	Output Shape	Param #
=====		
lstm (LSTM)	(None, 50)	10400
dense_15 (Dense)	(None, 1)	51
=====		
Total params: 10451 (40.82 KB)		
Trainable params: 10451 (40.82 KB)		
Non-trainable params: 0 (0.00 Byte)		

```
In [43]: # testing
full = np.concatenate((train, test))
test_days = pd.DataFrame(full)[len(full) - len(test) - prev_months:].values
generator_test = timeseries_dataset_from_array(
    test_days,
    test_days,
    sequence_length=prev_months)

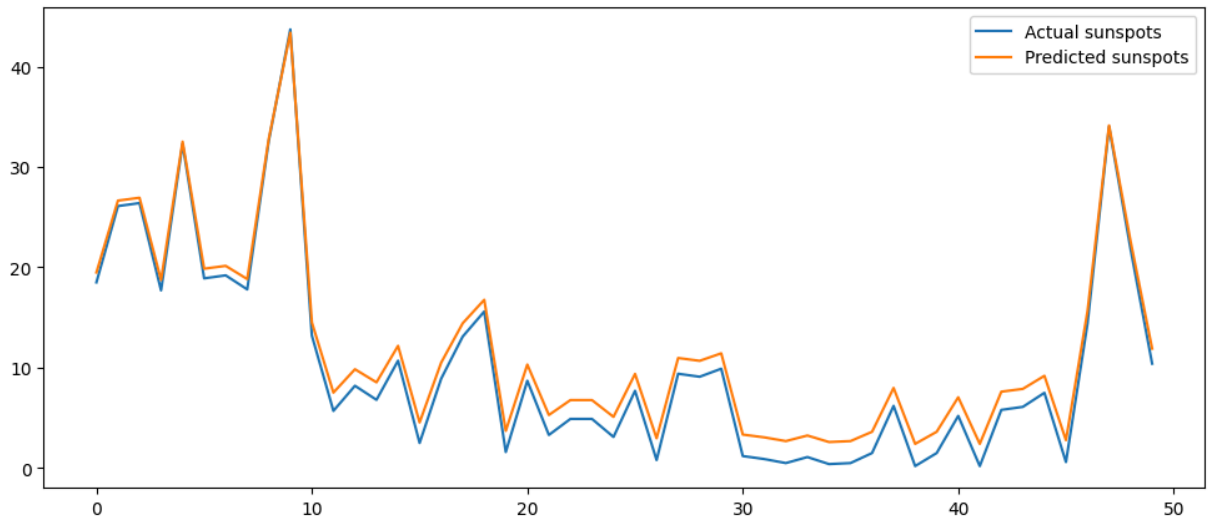
# Predicciones
preds = model.predict(generator_test)

1/1 [=====] - 0s 237ms/step
```

```
In [44]: plt.figure(figsize = (12, 5))
plt.plot(scaler.inverse_transform(test_days), label='Actual sunspots')
```

```
plt.plot(scaler.inverse_transform(preds), label='Predicted sunspots')
plt.legend()
```

Out[44]: <matplotlib.legend.Legend at 0x7fae46a562f0>



Redes Neuronales Convolucionales (CNN-Convolutional Neural Networks)

- Así como las redes neuronales recurrentes permiten procesar datos temporalmente relacionados, las redes neuronales convolucionales permiten procesar datos espacialmente relacionados.
- Este tipo de redes son utilizadas generalmente para procesar imágenes, donde los píxeles están relacionados espacialmente entre sí.
- En las redes convolucionales, generalmente se utilizan **capas de pooling**. Estas capas se utilizan para reducción de dimensionalidad, disminuyendo la redundancia de datos existente en imágenes (píxeles cercanos generalmente están altamente correlacionados).

Convolución

- Sean dos funciones $f()$ y $g()$, se define la operación de convolución entre ellas de la siguiente manera

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) \cdot g(t - \tau) d\tau$$

De esta operación se obtiene una nueva función resultante de operar sobre la función $f()$ con la función $g()$ mientras esta se traslada en el tiempo.

- La convolución es una operación muy utilizada en el filtrado de señales donde un filtro $g()$ se aplica sobre una señal $f()$

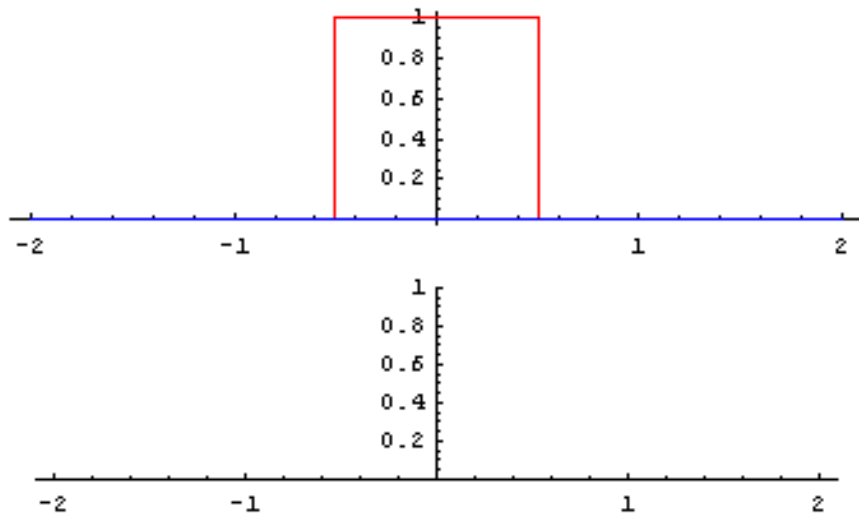
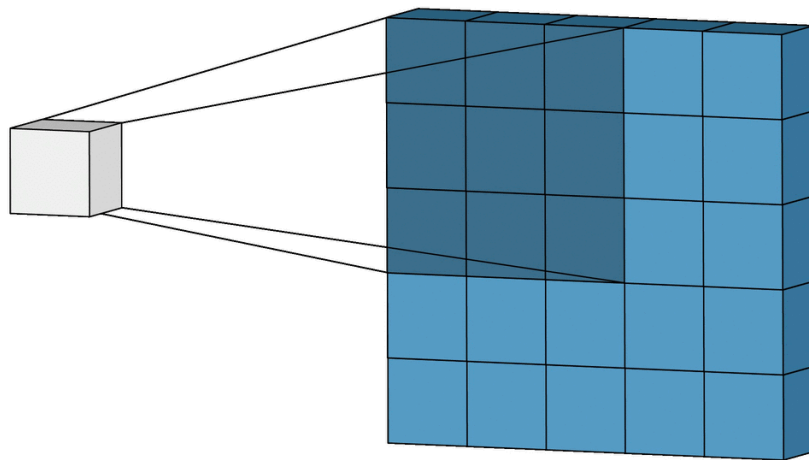


Imagen Licencia CC

- Las redes convolucionales son redes que contienen capas de procesamiento convolucional.
- Por lo general este tipo de redes se utiliza para el procesamiento de imágenes, donde se tiene un conjunto de filtros convolucionales bi-dimensionales aplicado sobre una imagen.



Fuente: <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

Ejemplos de formas de aplicar convolución:

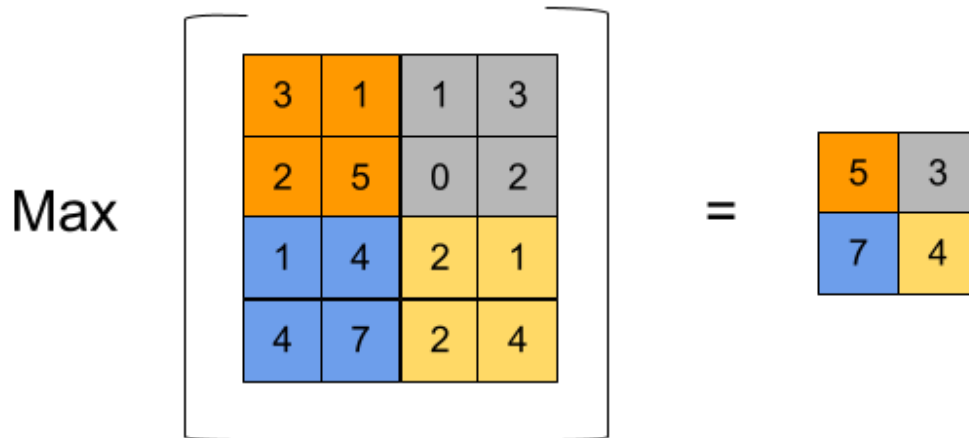
<https://hannibunny.github.io/mlbook/neuralnetworks/convolutionDemos.html>

Capas de Pooling

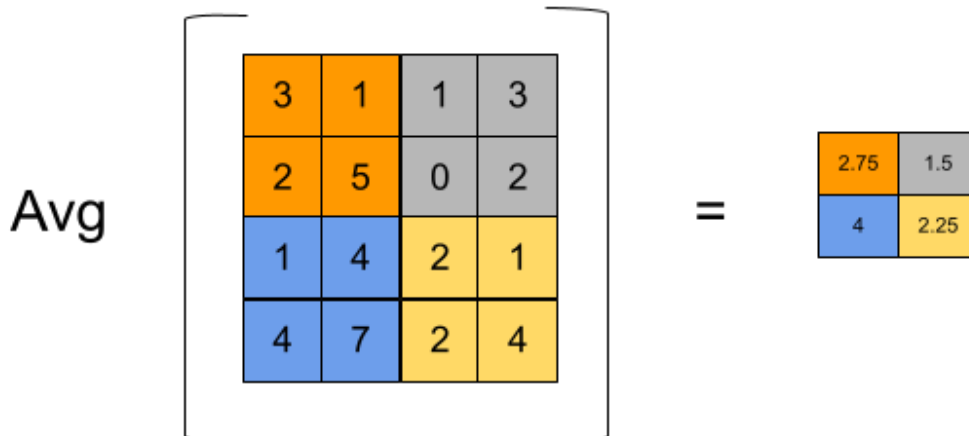
- Las capas de Pooling permiten reducir la dimensionalidad y la varianza espacial de las imágenes en la medida que avanzan por las capas de la red.
- Esto permite reconocer patrones más generales en la imagen.

Tipos de Pooling

- Max Pooling: Se obtiene el máximo de cada zona



- Average Pooling: Se obtiene el promedio de cada zona



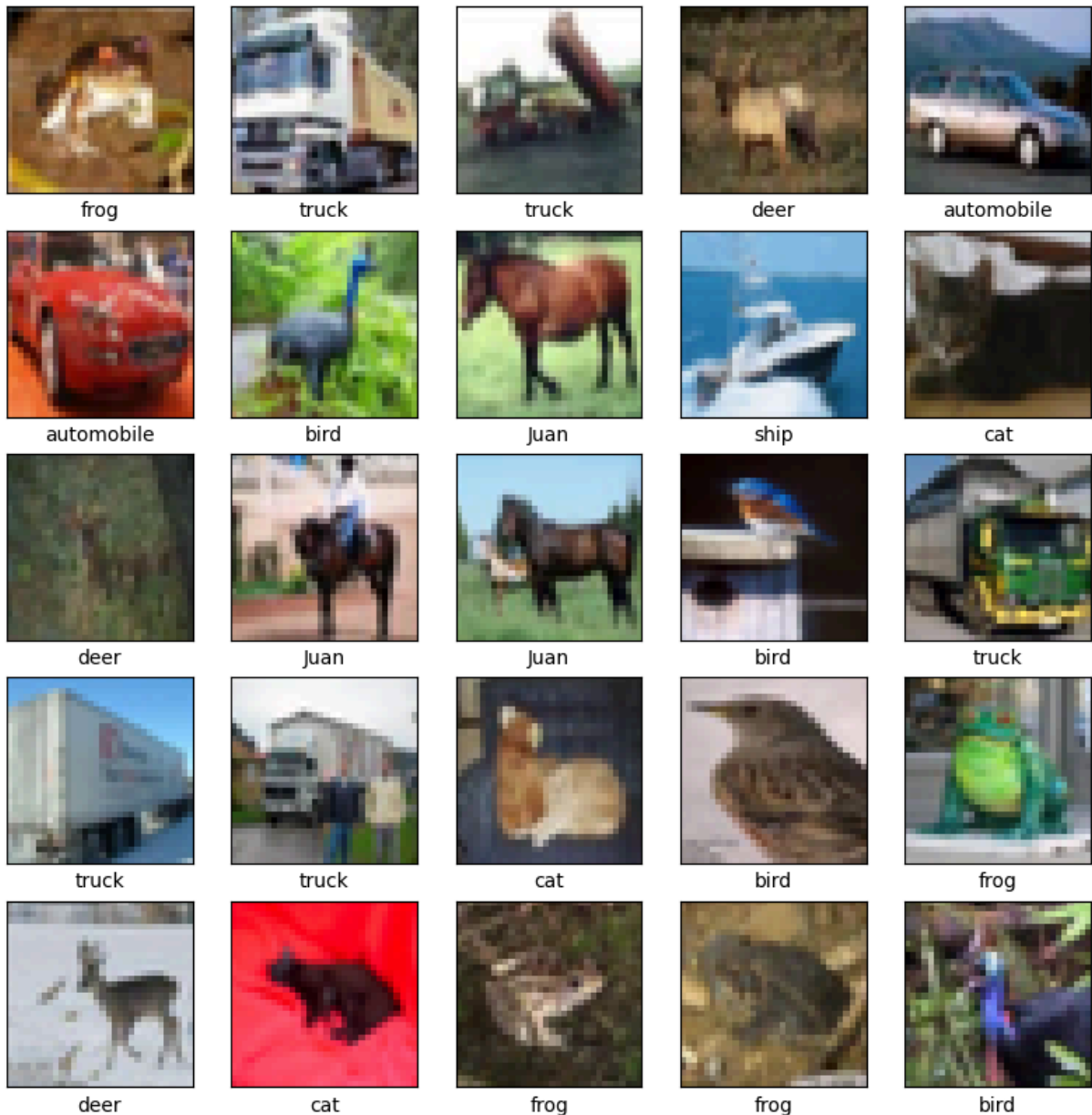
```
In [45]: # Ejemplo, fuente https://www.tensorflow.org/tutorials/images/cnn
# Bibliotecas
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

```
In [46]: # Datos
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'Juan', 'ship', 'truck']
```

```
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



```
In [47]: print(train_images.shape)
print(test_images.shape)
print(type(train_images[0]))
```

```
(50000, 32, 32, 3)
(10000, 32, 32, 3)
<class 'numpy.ndarray'>
```

```
In [48]: # Creación de modelo
model = models.Sequential()
# Capas encargadas de obtener información de la imagen
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32,
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
# Capas para la clasificación en base a la información obtenida en
# capas anteriores
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36928
flatten_1 (Flatten)	(None, 1024)	0
dense_16 (Dense)	(None, 64)	65600
dense_17 (Dense)	(None, 10)	650
Total params: 122570 (478.79 KB)		
Trainable params: 122570 (478.79 KB)		
Non-trainable params: 0 (0.00 Byte)		

```
In [49]: model.compile(optimizer='sgd',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])
history_cnn = model.fit(train_images, train_labels, epochs=30,
                        #validation_data=(test_images, test_labels))
```

Epoch 1/30
1563/1563 [=====] - 6s 4ms/step - loss: 2.0368 - accuracy: 0.2517
Epoch 2/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.7031 - accuracy: 0.3847
Epoch 3/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.5199 - accuracy: 0.4513
Epoch 4/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.3977 - accuracy: 0.4983
Epoch 5/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.3052 - accuracy: 0.5338
Epoch 6/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.2318 - accuracy: 0.5650
Epoch 7/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.1684 - accuracy: 0.5875
Epoch 8/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.1106 - accuracy: 0.6072
Epoch 9/30
1563/1563 [=====] - 8s 5ms/step - loss: 1.0626 - accuracy: 0.6263
Epoch 10/30
1563/1563 [=====] - 6s 4ms/step - loss: 1.0147 - accuracy: 0.6451
Epoch 11/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.9732 - accuracy: 0.6620
Epoch 12/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.9391 - accuracy: 0.6700
Epoch 13/30
1563/1563 [=====] - 7s 4ms/step - loss: 0.9058 - accuracy: 0.6841
Epoch 14/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.8743 - accuracy: 0.6956
Epoch 15/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.8437 - accuracy: 0.7038
Epoch 16/30
1563/1563 [=====] - 8s 5ms/step - loss: 0.8146 - accuracy: 0.7177
Epoch 17/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.7884 - accuracy: 0.7240
Epoch 18/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.7630 - accuracy: 0.7338
Epoch 19/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.7368 - accuracy: 0.7438


```

curacy: 0.7434
Epoch 20/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.7124 - ac
curacy: 0.7514
Epoch 21/30
1563/1563 [=====] - 5s 3ms/step - loss: 0.6893 - ac
curacy: 0.7597
Epoch 22/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.6697 - ac
curacy: 0.7672
Epoch 23/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.6444 - ac
curacy: 0.7745
Epoch 24/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.6240 - ac
curacy: 0.7823
Epoch 25/30
1563/1563 [=====] - 7s 4ms/step - loss: 0.6013 - ac
curacy: 0.7889
Epoch 26/30
1563/1563 [=====] - 8s 5ms/step - loss: 0.5829 - ac
curacy: 0.7947
Epoch 27/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.5626 - ac
curacy: 0.8016
Epoch 28/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.5417 - ac
curacy: 0.8086
Epoch 29/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.5238 - ac
curacy: 0.8177
Epoch 30/30
1563/1563 [=====] - 6s 4ms/step - loss: 0.5059 - ac
curacy: 0.8232

```

```

In [50]: #plt.plot(history_cnn.history['accuracy'], label='train_accuracy')
#plt.plot(history_cnn.history['val_accuracy'], label = 'val_accuracy')
#plt.xlabel('Epoch')
#plt.ylabel('Accuracy')
#plt.ylim([0.5, 1])
#plt.legend(loc='lower right')

test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

```

```

313/313 - 1s - loss: 1.0163 - accuracy: 0.6796 - 536ms/epoch - 2ms/step

```

```

In [51]: index = 60
lala = model.predict(test_images[index].reshape(-1,32,32,3))
import numpy as np
print(class_names[test_labels[index][0]])
print(class_names[np.argmax(lala)])

```

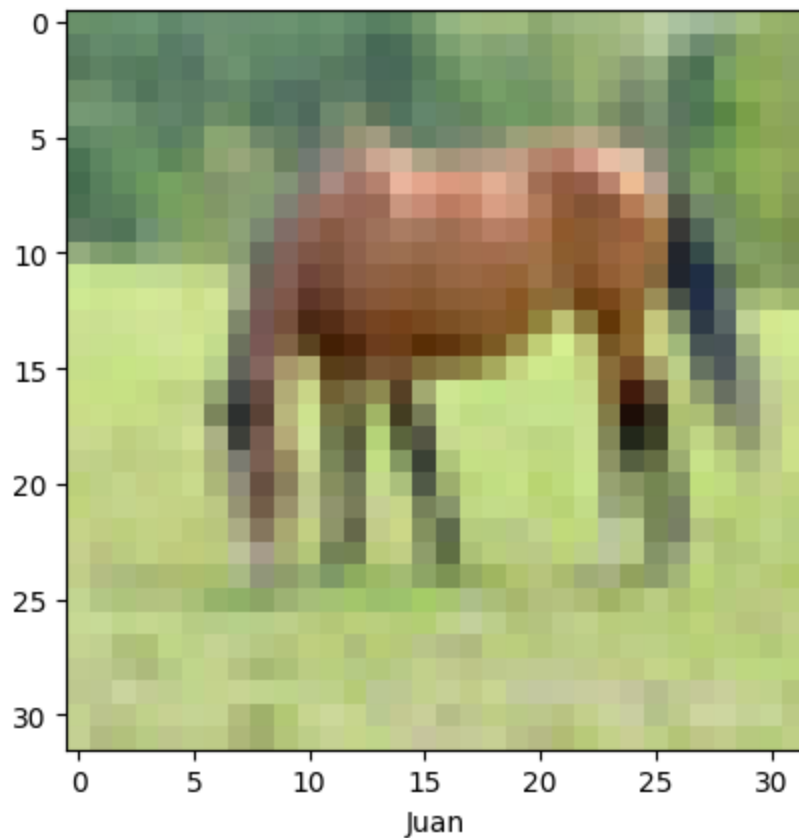
```

1/1 [=====] - 0s 55ms/step

```

Juan
Juan

```
In [52]: plt.grid(False)
plt.imshow(test_images[index])
plt.xlabel(class_names[test_labels[index][0]])
plt.show()
```



```
In [53]: from IPython.core.display import HTML
HTML("""
<style>
.output_png {
    display: table-cell;
    text-align: center;
    vertical-align: middle;
}
</style>
""")
#codigo extra, para que imagenes de matplotlib
#estén centradas en las diapositivas, ejecutar antes de lanzar los ejemplos.
```

Out[53]: